Surya OCR to extract data for fine-tuning

In [14]:
```
!pip install transformers surya-ocr
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.6)
Requirement already satisfied: surya-ocr in /usr/local/lib/python3.12/dist-packages (0.17.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.24.3)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transfo
rmers) (0.36.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (202
5.11.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transfor
mers) (0.22.2)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.
7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.12/dist-packages (from surya-ocr) (8.3.
1)
Requirement already satisfied: einops<0.9.0,>=0.8.1 in /usr/local/lib/python3.12/dist-packages (from surya-ocr) (0.
8.1)
Requirement already satisfied: filetype<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(1.2.0)
Requirement already satisfied: opencv-python-headless==4.11.0.86 in /usr/local/lib/python3.12/dist-packages (from su
rya-ocr) (4.11.0.86)
Requirement already satisfied: pillow<11.0.0,>=10.2.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr) (1
0.4.0)
Requirement already satisfied: platformdirs<5.0.0,>=4.3.6 in /usr/local/lib/python3.12/dist-packages (from surya-oc
r) (4.5.1)
Requirement already satisfied: pre-commit<5.0.0,>=4.2.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(4.5.1)
Requirement already satisfied: pydantic<3.0.0,>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(2.12.3)
Requirement already satisfied: pydantic-settings<3.0.0,>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from sury
a-ocr) (2.12.0)
Requirement already satisfied: pypdfium2==4.30.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr) (4.30.
0)
Requirement already satisfied: python-dotenv<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from surya-oc
r) (1.2.1)
Requirement already satisfied: torch<3.0.0,>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr) (2.9.
0+cpu)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.
```

```
0,>=0.34.0->transformers) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub
<1.0,>=0.34.0->transformers) (1.2.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingfa
ce-hub<1.0,>=0.34.0->transformers) (4.15.0)
Requirement already satisfied: cfgv>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.0,>=4.2.0
->surya-ocr) (3.5.0)
Requirement already satisfied: identify>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.0,>=
4.2.0->surya-ocr) (2.6.16)
Requirement already satisfied: nodeenv>=0.11.1 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.0,>=
4.2.0->surya-ocr) (1.10.0)
Requirement already satisfied: virtualenv>=20.10.0 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.
0,>=4.2.0->surya-ocr) (21.0.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.
0,>=2.5.3->surya-ocr) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.
0,>=2.5.3->surya-ocr) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.
0.0,>=2.5.3->surya-ocr) (0.4.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0->sury
a-ocr) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0->s
urya-ocr) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0-
>surya-ocr) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0->surya-oc
r) (3.1.6)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->t
ransformers) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers)
(3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transfo
rmers) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transfo
rmers) (2026.1.4)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->to
rch<3.0.0,>=2.7.0->surya-ocr) (1.3.0)
Requirement already satisfied: distlib<1,>=0.3.7 in /usr/local/lib/python3.12/dist-packages (from virtualenv>=20.10.
0->pre-commit<5.0.0,>=4.2.0->surya-ocr) (0.4.0)
Requirement already satisfied: python-discovery>=1 in /usr/local/lib/python3.12/dist-packages (from virtualenv>=20.1
0.0->pre-commit<5.0.0,>=4.2.0->surya-ocr) (1.1.0)
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch<3.0.0,
>=2.7.0->surya-ocr) (3.0.3)
```

In [15]:
```
!pip install 'optimum[onnxruntime]'
```

```
Requirement already satisfied: optimum[onnxruntime] in /usr/local/lib/python3.12/dist-packages (2.1.0)
Requirement already satisfied: transformers>=4.29 in /usr/local/lib/python3.12/dist-packages (from optimum[onnxrunti
me]) (4.57.6)
Requirement already satisfied: torch>=1.11 in /usr/local/lib/python3.12/dist-packages (from optimum[onnxruntime])
(2.9.0+cpu)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from optimum[onnxruntime]) (25.
0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from optimum[onnxruntime]) (2.0.2)
Requirement already satisfied: huggingface_hub>=0.8.0 in /usr/local/lib/python3.12/dist-packages (from optimum[onnxr
untime]) (0.36.2)
Requirement already satisfied: optimum-onnx[onnxruntime] in /usr/local/lib/python3.12/dist-packages (from optimum[on
nxruntime]) (0.1.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.8.0->opt
imum[onnxruntime]) (3.24.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.
8.0->optimum[onnxruntime]) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub
>=0.8.0->optimum[onnxruntime]) (1.2.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.8.0->
optimum[onnxruntime]) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.8.0->opt
imum[onnxruntime]) (2.32.4)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.8.0-
>optimum[onnxruntime]) (4.67.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingfa
ce_hub>=0.8.0->optimum[onnxruntime]) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.11->optimum[onnx
runtime]) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11->optimum[o
nnxruntime]) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11->optimum
[onnxruntime]) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.11->optimum[onnxrunt
ime]) (3.1.6)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.29
->optimum[onnxruntime]) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transfor
mers>=4.29->optimum[onnxruntime]) (0.22.2)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.2
9->optimum[onnxruntime]) (0.7.0)
Requirement already satisfied: onnx in /usr/local/lib/python3.12/dist-packages (from optimum-onnx[onnxruntime]; extr
```

```
a == "onnxruntime"->optimum[onnxruntime]) (1.20.1)
Requirement already satisfied: onnxruntime>=1.18.0 in /usr/local/lib/python3.12/dist-packages (from optimum-onnx[onn
xruntime]; extra == "onnxruntime"->optimum[onnxruntime]) (1.24.2)
Requirement already satisfied: flatbuffers in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.18.0->opt
imum-onnx[onnxruntime]; extra == "onnxruntime"->optimum[onnxruntime]) (25.12.19)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.18.0->optimu
m-onnx[onnxruntime]; extra == "onnxruntime"->optimum[onnxruntime]) (5.29.5)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->to
rch>=1.11->optimum[onnxruntime]) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.11-
>optimum[onnxruntime]) (3.0.3)
Requirement already satisfied: ml_dtypes>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from onnx->optimum-onnx
[onnxruntime]; extra == "onnxruntime"->optimum[onnxruntime]) (0.5.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->h
uggingface_hub>=0.8.0->optimum[onnxruntime]) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->huggingface_h
ub>=0.8.0->optimum[onnxruntime]) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->hugging
face_hub>=0.8.0->optimum[onnxruntime]) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->hugging
face_hub>=0.8.0->optimum[onnxruntime]) (2026.1.4)
```

In [16]:
```
!pip install surya-ocr --upgrade
```

Out[16]:
```
['Requirement already satisfied: surya-ocr in /usr/local/lib/python3.12/dist-packages (0.17.1)',
 'Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(8.3.1)',
 'Requirement already satisfied: einops<0.9.0,>=0.8.1 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(0.8.1)',
 'Requirement already satisfied: filetype<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from surya-oc
r) (1.2.0)',
 'Requirement already satisfied: opencv-python-headless==4.11.0.86 in /usr/local/lib/python3.12/dist-packages (fro
m surya-ocr) (4.11.0.86)',
 'Requirement already satisfied: pillow<11.0.0,>=10.2.0 in /usr/local/lib/python3.12/dist-packages (from surya-oc
r) (10.4.0)',
 'Requirement already satisfied: platformdirs<5.0.0,>=4.3.6 in /usr/local/lib/python3.12/dist-packages (from surya
-ocr) (4.5.1)',
 'Requirement already satisfied: pre-commit<5.0.0,>=4.2.0 in /usr/local/lib/python3.12/dist-packages (from surya-o
cr) (4.5.1)',
 'Requirement already satisfied: pydantic<3.0.0,>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from surya-oc
r) (2.12.3)',
 'Requirement already satisfied: pydantic-settings<3.0.0,>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from
surya-ocr) (2.12.0)',
 'Requirement already satisfied: pypdfium2==4.30.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr) (4.
30.0)',
 'Requirement already satisfied: python-dotenv<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from sury
a-ocr) (1.2.1)',
 'Requirement already satisfied: torch<3.0.0,>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(2.9.0+cpu)',
 'Requirement already satisfied: transformers>=4.56.1 in /usr/local/lib/python3.12/dist-packages (from surya-ocr)
(4.57.6)',
 'Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.12/dist-packages (from opencv-python-head
less==4.11.0.86->surya-ocr) (2.0.2)',
 'Requirement already satisfied: cfgv>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.0,>=
4.2.0->surya-ocr) (3.5.0)',
 'Requirement already satisfied: identify>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.
0,>=4.2.0->surya-ocr) (2.6.16)',
 'Requirement already satisfied: nodeenv>=0.11.1 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.
0,>=4.2.0->surya-ocr) (1.10.0)',
 'Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from pre-commit<5.0.0,>=
4.2.0->surya-ocr) (6.0.3)',
 'Requirement already satisfied: virtualenv>=20.10.0 in /usr/local/lib/python3.12/dist-packages (from pre-commit<
5.0.0,>=4.2.0->surya-ocr) (21.0.0)',
 'Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<
```

```
3.0.0,>=2.5.3->surya-ocr) (0.7.0)',
 'Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<
3.0.0,>=2.5.3->surya-ocr) (2.41.4)',
 'Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydant
ic<3.0.0,>=2.5.3->surya-ocr) (4.15.0)',
 'Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydanti
c<3.0.0,>=2.5.3->surya-ocr) (0.4.2)',
 'Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0->su
rya-ocr) (3.24.3)',
 'Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0->
surya-ocr) (75.2.0)',
 'Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.
0->surya-ocr) (1.14.0)',
 'Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.
7.0->surya-ocr) (3.6.1)',
 'Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.0->sury
a-ocr) (3.1.6)',
 'Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch<3.0.0,>=2.7.
0->surya-ocr) (2025.3.0)',
 'Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from tra
nsformers>=4.56.1->surya-ocr) (0.36.2)',
 'Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.
56.1->surya-ocr) (25.0)',
 'Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers>=
4.56.1->surya-ocr) (2025.11.3)',
 'Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers>=4.56.1->s
urya-ocr) (2.32.4)',
 'Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from tran
sformers>=4.56.1->surya-ocr) (0.22.2)',
 'Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers>
=4.56.1->surya-ocr) (0.7.0)',
 'Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers>=4.56.1-
>surya-ocr) (4.67.1)',
 'Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface
-hub<1.0,>=0.34.0->transformers>=4.56.1->surya-ocr) (1.2.0)',
 'Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3
->torch<3.0.0,>=2.7.0->surya-ocr) (1.3.0)',
 'Requirement already satisfied: distlib<1,>=0.3.7 in /usr/local/lib/python3.12/dist-packages (from virtualenv>=2
0.10.0->pre-commit<5.0.0,>=4.2.0->surya-ocr) (0.4.0)',
 'Requirement already satisfied: python-discovery>=1 in /usr/local/lib/python3.12/dist-packages (from virtualenv>=
20.10.0->pre-commit<5.0.0,>=4.2.0->surya-ocr) (1.1.0)',
```

```
 'Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch<3.
0.0,>=2.7.0->surya-ocr) (3.0.3)',
 'Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
s->transformers>=4.56.1->surya-ocr) (3.4.4)',
 'Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transform
ers>=4.56.1->surya-ocr) (3.11)',
 'Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->tra
nsformers>=4.56.1->surya-ocr) (2.5.0)',
 'Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->tra
nsformers>=4.56.1->surya-ocr) (2026.1.4)']
```

In [17]:

```python
import os
import gc
import torch
from PIL import Image
from surya.detection import DetectionPredictor
from surya.layout import LayoutPredictor
from surya.foundation import FoundationPredictor
from surya.settings import settings

# --- MEMORY CONFIGURATION ---
os.environ["DETECTOR_BATCH_SIZE"] = "2"
os.environ["LAYOUT_BATCH_SIZE"] = "2"

def get_layout_map(image_path):
    """Identifies structural blocks and clears memory immediately after."""
    img = Image.open(image_path).convert("RGB")
    fp = FoundationPredictor(checkpoint=settings.LAYOUT_MODEL_CHECKPOINT)
    lp = LayoutPredictor(fp)

    layout_results = lp([img])[0]

    # Sort by 'position' to maintain reading flow
    sorted_blocks = sorted(layout_results.bboxes, key=lambda x: x.position)

    # Explicit Cleanup to prevent RAM crashes
    del lp, fp
    gc.collect()
    torch.cuda.empty_cache() if torch.cuda.is_available() else None
```

```
    return sorted_blocks
```

In [26]:
```
from PIL import Image

def filter_layout_blocks(image_path, raw_blocks):
    """
    Filters blocks to eliminate side text (marginalia) and
    handles coordinate padding for accepted blocks.
    """
    img = Image.open(image_path)
    width, height = img.size

    # Target labels for the main content
    TARGET_LABELS = ["text", "sectionheader", "title", "section-header", "section_header","pageheader"]
    BOTTOM_PADDING = 20

    accepted_blocks = []
    rejected_blocks = []

    for block in raw_blocks:
        # 1. Extract coordinates and clean label
        x1, y1, x2, y2 = block.bbox
        clean_label = block.label.lower().replace(" ", "").replace("_", "").replace("-", "")

        # --- FIXED: Create block_dict FIRST ---
        # This converts the Surya object into a dictionary you can modify
        block_dict = block.model_dump()

        # 2. Logic to eliminate side text
        # Marginalia usually starts in the far right margin (e.g., > 80% width)
        is_not_too_far_right = x1 < (width * 0.80)

        # Main blocks are typically wider than 20% of the page
        is_wide_enough = (x2 - x1) > (width * 0.15)

        # Explicitly reject labels Surya often gives to side notes
        is_not_marginal_label = clean_label not in ["caption", "footnote", "formula"]

        # 3. Handle the "Drop-Cap" Ornate Letter Exception
```

```
        # Keeps the decorative 'A' if it's on the far left
        is_drop_cap = (clean_label == "figure" and x1 < (width * 0.25))

        # 4. Final Decision Gate
        if (clean_label in TARGET_LABELS and is_not_too_far_right and is_wide_enough and is_not_marginal_label) or
            # Apply bottom padding to avoid clipping text descenders
            block_dict['bbox'][3] = min(height, y2 + BOTTOM_PADDING)
            accepted_blocks.append(block_dict)
        else:
            # Rejects biblical citations and marginalia
            rejected_blocks.append(block_dict)

    print(f"Total Raw Blocks: {len(raw_blocks)}")
    print(f"Accepted Blocks: {len(accepted_blocks)}")
    print(f"Rejected Blocks: {len(rejected_blocks)}")

    return accepted_blocks, rejected_blocks
```

In [28]:

```
from PIL import Image
from IPython.display import display

def view_accepted_images(image_path, accepted_blocks):
    """
    Displays the cropped images for each block.
    Works with both Surya objects and dictionary-formatted blocks.
    """
    img = Image.open(image_path).convert("RGB")

    print(f"--- Displaying {len(accepted_blocks)} Accepted Blocks ---")

    for i, block in enumerate(accepted_blocks):
        # Handle dictionary access
        if isinstance(block, dict):
            crop_box = block['bbox']
            label = block.get('label', 'Unknown')
            pos = block.get('position', i)
        else:
            # Fallback for original Surya objects
            crop_box = block.bbox
            label = block.label
```

```
            pos = block.position

        # Crop the image to the block's boundaries
        cropped_img = img.crop(crop_box)

        print(f"Block {i} | Label: {label} | Position: {pos}")
        display(cropped_img)
        print("-" * 30)

# Run this to see your blocks
# view_accepted_images(image_path, rejected_blocks)
```

In [29]:

```
import os
import gc
import torch
from PIL import Image
from surya.detection import DetectionPredictor

def merge_nearby_lines(bboxes, x_threshold=100):
    """
    Groups bboxes that are on the same vertical level (y) and
    close horizontally (x) to fix split titles.
    """
    if not bboxes: return []
    # Sort primarily by Top Y, then by Left X
    sorted_bboxes = sorted(bboxes, key=lambda b: (b.bbox[1], b.bbox[0]))

    merged = []
    if sorted_bboxes:
        current = list(sorted_bboxes[0].bbox)
        for next_box in sorted_bboxes[1:]:
            nx1, ny1, nx2, ny2 = next_box.bbox
            # Check if they overlap vertically and are close horizontally
            vertical_mid = (ny1 + ny2) / 2
            if (current[1] <= vertical_mid <= current[3]) and (nx1 - current[2] < x_threshold):
                # Merge the boxes
                current[0] = min(current[0], nx1)
                current[1] = min(current[1], ny1)
                current[2] = max(current[2], nx2)
                current[3] = max(current[3], ny2)
```

```
            else:
                merged.append(current)
                current = list(next_box.bbox)
        merged.append(current)
    return merged

def extract_surgical_lines_v6(image_path, accepted_blocks, output_folder="final_dataset"):
    full_img = Image.open(image_path).convert("RGB")
    width, height = full_img.size
    det_predictor = DetectionPredictor()
    os.makedirs(output_folder, exist_ok=True)

    line_counter = 0
    for block in accepted_blocks:
        crop_box = block['bbox']
        block_crop = full_img.crop(crop_box)

        # Get the clean label for logic check
        label = block.get('label', 'text').lower().replace(" ", "").replace("_", "").replace("-", "")

        # --- NEW LOGIC: Conditional Line Detection ---
        if "sectionheader" in label or "title" in label:
            # Skip line detection: Save the entire block as one image
            file_name = f"line_{line_counter:04d}_{label}_full.png"
            block_crop.save(os.path.join(output_folder, file_name))
            line_counter += 1
        else:
            # Standard Text Logic: Perform line detection and merging
            line_results = det_predictor([block_crop])[0]

            # Merge split words (using your 15% threshold)
            merged_bboxes = merge_nearby_lines(line_results.bboxes, x_threshold=width * 0.15)

            for bbox in merged_bboxes:
                lx1, ly1, lx2, ly2 = bbox
                l_width = lx2 - lx1
                l_height = ly2 - ly1

                # Skip small symbols/noise
                if l_width < 40 and l_height < 40:
                    continue
```

```
                line_img = block_crop.crop(bbox)
                file_name = f"line_{line_counter:04d}_{label}.png"
                line_img.save(os.path.join(output_folder, file_name))
                line_counter += 1

    # Cleanup
    del det_predictor
    gc.collect()
    torch.cuda.empty_cache() if torch.cuda.is_available() else None
    return line_counter
```

In [30]:
```
import gc
import torch

def clear_memory():
    """Clears Python garbage collector and CUDA cache to prevent OOM."""
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    print("Memory cleared.")
```

Pages with transcriptions given in the dataset

```
{
    "Buendia - Instruccion transcription":[2,3,4],
    "Covarrubias - Tesoro lengua transcription":[7,8,9],
    "Guardiola - Tratado nobleza transcription":[12,13,14],
    "PORCONES.23.5 - 1628 transcription":[1,2,3,4(left)],
    "PORCONES.228.38 - 1646 transcription":[1,2,3,4,5],
    "PORCONES.748.6 - 1650 Transcription":[1,2,3,4]
}
```

In [ ]:
```
import os
import gc
import json
import torch
Image.MAX_IMAGE_PIXELS = None
```

```
GT_PAGES_PATH = "/kaggle/input/datasets/indrasn0wal/maetaskocr/kaggle/working/gt_pages"
SURYA_LINES_PATH = "/kaggle/working/lines_detected_surya"
LAYOUT_MAP_PATH = "/kaggle/working/layout_maps.json"

os.makedirs(SURYA_LINES_PATH, exist_ok=True)
# This is used to skip file pages. I used because, I was processing the files twice, so already processed files wer
# SKIP_FILES = {
#     "Buendia_-_Instruccion": ["p2_full.jpg", "p2_left.jpg", "p3_left.jpg", "p3_right.jpg", "p4_left.jpg", "p4_rig
#     "Covarrubias_-_Tesoro_lengua":["p7_full.jpg","p8_full.jpg","p9_full.jpg"],
#     "Guardiola_-_Tratado_nobleza":["p12_full.jpg","p13_full.jpg","p14_full.jpg"],
#     "PORCONES.23.5_-_1628":  ["p4_right.jpg"]
# }

all_layout_maps = {}
total_lines = 0

for source_folder in sorted(os.listdir(GT_PAGES_PATH)):
    source_in = os.path.join(GT_PAGES_PATH, source_folder)
    if not os.path.isdir(source_in):
        continue

    source_out = os.path.join(SURYA_LINES_PATH, source_folder)
    os.makedirs(source_out, exist_ok=True)

    for fname in sorted(os.listdir(source_in)):
        if not fname.endswith('.jpg'):
            continue
        # if fname in SKIP_FILES.get(source_folder, []):
        #     print(f"Skipped: {source_folder}/{fname}")
        #     continue

        # Create a sub-subfolder named after the image file
        # This turns 'p1.jpg' into a folder named 'p1'
        file_slug = fname.replace('.jpg', '')
        source_out = os.path.join(SURYA_LINES_PATH, source_folder, file_slug)
        os.makedirs(source_out, exist_ok=True)

        image_path = os.path.join(source_in, fname)
        page_key = f"{source_folder}/{fname}"
        print(f"\nProcessing: {page_key}")
```

```
        raw_layout_blocks = get_layout_map(image_path)
        clear_memory()

        accepted_blocks, rejected_blocks = filter_layout_blocks(image_path, raw_layout_blocks)

        all_layout_maps[page_key] = {
            "accepted": accepted_blocks,
            "rejected": [b.model_dump() if hasattr(b, 'model_dump') else b for b in rejected_blocks],
            "total_raw": len(raw_layout_blocks),
            "total_accepted": len(accepted_blocks),
            "total_rejected": len(rejected_blocks)
        }

        n_lines = extract_surgical_lines_v6(image_path, accepted_blocks, output_folder=source_out)
        print(f"  Accepted: {len(accepted_blocks)} blocks | Lines: {n_lines}")
        total_lines += n_lines
        clear_memory()

with open(LAYOUT_MAP_PATH, 'w', encoding='utf-8') as f:
    json.dump(all_layout_maps, f, ensure_ascii=False, indent=2)

print(f"\nTotal lines extracted: {total_lines}")
print(f"Layout maps saved to: {LAYOUT_MAP_PATH}")
```

```
Skipped: Buendia_-_Instruccion/p2_full.jpg
Skipped: Buendia_-_Instruccion/p2_left.jpg
Skipped: Buendia_-_Instruccion/p2_right.jpg
Skipped: Buendia_-_Instruccion/p3_full.jpg
Skipped: Buendia_-_Instruccion/p3_left.jpg
Skipped: Buendia_-_Instruccion/p3_right.jpg
Skipped: Buendia_-_Instruccion/p4_full.jpg
Skipped: Buendia_-_Instruccion/p4_left.jpg
Skipped: Buendia_-_Instruccion/p4_right.jpg
Skipped: Covarrubias_-_Tesoro_lengua/p7_full.jpg
Skipped: Covarrubias_-_Tesoro_lengua/p8_full.jpg
Skipped: Covarrubias_-_Tesoro_lengua/p9_full.jpg
Skipped: Guardiola_-_Tratado_nobleza/p12_full.jpg
Skipped: Guardiola_-_Tratado_nobleza/p13_full.jpg
Skipped: Guardiola_-_Tratado_nobleza/p14_full.jpg

Processing: PORCONES.228.38_-_1646/p1_full.jpg
```

```
Recognizing Layout:   0%|          | 0/1 [00:00<?, ?it/s]


Recognizing Layout: 100%|██████████| 1/1 [01:03<00:00, 63.53s/it]
Recognizing Layout:   0%|          | 0/1 [34:29<?, ?it/s]
Recognizing Layout:   0%|          | 0/1 [04:01<?, ?it/s]
Memory cleared.
Total Raw Blocks: 10
Accepted Blocks: 9
Rejected Blocks: 1
Detecting bboxes: 100%|██████████| 1/1 [00:05<00:00,  5.13s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.38s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.55s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:09<00:00,  9.37s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.41s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:08<00:00,  8.48s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.37s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:16<00:00, 16.62s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.35s/it]
  Accepted: 9 blocks | Lines: 21
Memory cleared.

Processing: PORCONES.228.38_-_1646/p2_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:02<00:00, 62.09s/it]
Memory cleared.
Total Raw Blocks: 7
Accepted Blocks: 5
Rejected Blocks: 2
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.55s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:21<00:00, 21.74s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:32<00:00, 32.73s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:13<00:00, 13.07s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.97s/it]
  Accepted: 5 blocks | Lines: 32
Memory cleared.

Processing: PORCONES.228.38_-_1646/p3_full.jpg
```

```
Recognizing Layout: 100%|██████████| 1/1 [01:15<00:00, 75.34s/it]
Memory cleared.
Total Raw Blocks: 9
Accepted Blocks: 6
Rejected Blocks: 3
Detecting bboxes: 100%|██████████| 1/1 [00:12<00:00, 12.83s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.89s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:17<00:00, 17.36s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:08<00:00,  8.95s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:16<00:00, 16.66s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.42s/it]
  Accepted: 6 blocks | Lines: 37
Memory cleared.

Processing: PORCONES.228.38_-_1646/p4_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:02<00:00, 62.77s/it]
Memory cleared.
Total Raw Blocks: 8
Accepted Blocks: 7
Rejected Blocks: 1
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.67s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:13<00:00, 13.50s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.64s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:21<00:00, 21.26s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:12<00:00, 12.91s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.50s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:15<00:00, 15.31s/it]
  Accepted: 7 blocks | Lines: 36
Memory cleared.

Processing: PORCONES.228.38_-_1646/p5_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:01<00:00, 61.21s/it]
Memory cleared.
Total Raw Blocks: 4
Accepted Blocks: 2
Rejected Blocks: 2
Detecting bboxes: 100%|██████████| 1/1 [00:29<00:00, 29.34s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:38<00:00, 38.08s/it]
```

```
  Accepted: 2 blocks | Lines: 35
Memory cleared.

Processing: PORCONES.23.5_-_1628/p1_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:03<00:00, 63.24s/it]
Memory cleared.
Total Raw Blocks: 15
Accepted Blocks: 8
Rejected Blocks: 7
Detecting bboxes: 100%|██████████| 1/1 [00:14<00:00, 14.04s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.70s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.74s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.42s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:09<00:00,  9.08s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:14<00:00, 14.05s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.53s/it]
  Accepted: 8 blocks | Lines: 24
Memory cleared.

Processing: PORCONES.23.5_-_1628/p2_left.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:00<00:00, 60.66s/it]
Memory cleared.
Total Raw Blocks: 10
Accepted Blocks: 8
Rejected Blocks: 2
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.46s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:26<00:00, 26.09s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.43s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.57s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.51s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.44s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:09<00:00,  9.07s/it]
  Accepted: 8 blocks | Lines: 30
Memory cleared.

Processing: PORCONES.23.5_-_1628/p2_right.jpg
Recognizing Layout: 100%|██████████| 1/1 [00:59<00:00, 59.82s/it]
```

```
Memory cleared.
Total Raw Blocks: 6
Accepted Blocks: 4
Rejected Blocks: 2
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.43s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.47s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:06<00:00,  6.50s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:47<00:00, 47.38s/it]
  Accepted: 4 blocks | Lines: 38
Memory cleared.

Processing: PORCONES.23.5_-_1628/p3_left.jpg
Recognizing Layout: 100%|██████████| 1/1 [00:59<00:00, 59.33s/it]
Memory cleared.
Total Raw Blocks: 4
Accepted Blocks: 3
Rejected Blocks: 1
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.49s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:39<00:00, 39.60s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:09<00:00,  9.75s/it]
  Accepted: 3 blocks | Lines: 36
Memory cleared.

Processing: PORCONES.23.5_-_1628/p3_right.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:01<00:00, 61.40s/it]
Memory cleared.
Total Raw Blocks: 5
Accepted Blocks: 3
Rejected Blocks: 2
Detecting bboxes: 100%|██████████| 1/1 [00:15<00:00, 15.56s/it]
Recognizing Layout: 100%|██████████| 1/1 [00:59<00:00, 59.68s/it]
Memory cleared.
Total Raw Blocks: 4
Accepted Blocks: 4
Rejected Blocks: 0
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.41s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:09<00:00,  9.71s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:21<00:00, 21.76s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:21<00:00, 21.28s/it]
```

```
  Accepted: 4 blocks | Lines: 35
Memory cleared.
Skipped: PORCONES.23.5_-_1628/p4_right.jpg

Processing: PORCONES.748.6_-_1650/p1_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:02<00:00, 62.52s/it]
Memory cleared.
Total Raw Blocks: 15
Accepted Blocks: 9
Rejected Blocks: 6
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.52s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:10<00:00, 10.08s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.41s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:15<00:00, 15.23s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.38s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:21<00:00, 21.58s/it]
  Accepted: 9 blocks | Lines: 30
Memory cleared.

Processing: PORCONES.748.6_-_1650/p2_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:02<00:00, 62.16s/it]
Memory cleared.
Total Raw Blocks: 6
Accepted Blocks: 5
Rejected Blocks: 1
Detecting bboxes: 100%|██████████| 1/1 [00:26<00:00, 26.67s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:09<00:00,  9.91s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.45s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:59<00:00, 59.55s/it]
  Accepted: 5 blocks | Lines: 34
Memory cleared.

Processing: PORCONES.748.6_-_1650/p3_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:01<00:00, 61.28s/it]
Memory cleared.
Total Raw Blocks: 4
Accepted Blocks: 1
Rejected Blocks: 3
Detecting bboxes: 100%|██████████| 1/1 [00:22<00:00, 22.26s/it]
```

```
  Accepted: 1 blocks | Lines: 8
Memory cleared.

Processing: PORCONES.748.6_-_1650/p4_full.jpg
Recognizing Layout: 100%|██████████| 1/1 [01:04<00:00, 64.09s/it]
Memory cleared.
Total Raw Blocks: 4
Accepted Blocks: 3
Rejected Blocks: 1
Detecting bboxes: 100%|██████████| 1/1 [01:00<00:00, 60.34s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:28<00:00, 28.12s/it]
Detecting bboxes: 100%|██████████| 1/1 [00:04<00:00,  4.78s/it]
  Accepted: 3 blocks | Lines: 32
Memory cleared.

Total lines extracted: 463
Layout maps saved to: /kaggle/working/layout_maps.json
```

In [46]:

```python
import json
import os
import docx
import re

TRANSCRIPTIONS_FIXED = "/kaggle/input/datasets/indrasn0wal/maetaskocr/kaggle/working/transcriptions_fixed"
MANUAL_REVIEW_PATH = "/kaggle/working/manual_review"
os.makedirs(MANUAL_REVIEW_PATH, exist_ok=True)

DOCX_MAP = {
    "Guardiola_-_Tratado_nobleza":  "Guardiola - Tratado nobleza transcription.docx",
    "PORCONES.23.5_-_1628":         "PORCONES.23.5 - 1628 transcription.docx",
    "PORCONES.228.38_-_1646":       "PORCONES.228.38 - 1646 transcription.docx",
    "PORCONES.748.6_-_1650":        "PORCONES.748.6 - 1650 Transcription.docx",
    "Covarrubias_-_Tesoro_lengua":  "Covarrubias - Tesoro lengua transcription.docx",
    "Buendia_-_Instruccion":        "Buendia - Instruccion transcription.docx",
}

PAGE_LABEL_PATTERN = re.compile(
    r"^PDF\s+p(\d+)(?:\s*[—-]\s*(left|right))?$",
    re.IGNORECASE
)
```

```python
STOP_PATTERNS = ["END OF EXTRACT", "NOTES:", "Notes:"]

def is_page_label(text):
    # Only match if text is a single line (no newlines)
    if '\n' in text:
        return False
    return bool(PAGE_LABEL_PATTERN.match(text.strip()))

def is_stop(text):
    first_line = text.split('\n')[0].strip()
    return any(first_line.startswith(s) for s in STOP_PATTERNS)

def parse_docx_lines(docx_path):
    doc = docx.Document(docx_path)
    pages = {}
    current_key = None
    found_first_page = False

    STOP_PATTERNS = ["END OF EXTRACT"]  # Remove NOTES: from stop patterns
    PAGE_LABEL_PATTERN = re.compile(
        r"^PDF\s+p(\d+)(?:\s*[—–]\s*(left|right))?$",
        re.IGNORECASE
    )

    for para in doc.paragraphs:
        text = para.text.strip()
        if not text:
            continue

        # Only stop at END OF EXTRACT after we've started reading
        if found_first_page and any(text.startswith(s) for s in STOP_PATTERNS):
            break

        # Skip NOTES block entirely — it appears before any page label
        if not found_first_page and text.startswith("NOTES:"):
            continue

        # Check if it's a page label
        match = PAGE_LABEL_PATTERN.match(text)
        if match:
```

```
            page_num = match.group(1)
            side = match.group(2)
            current_key = f"p{page_num}_{side.lower()}" if side else f"p{page_num}_full"
            pages[current_key] = []
            found_first_page = True
        elif current_key:
            for line in text.split('\n'):
                cleaned = ' '.join(line.split())
                if cleaned:
                    pages[current_key].append(cleaned)

    return pages

docx_splits = {}

for source_folder, docx_file in DOCX_MAP.items():
    docx_path = os.path.join(TRANSCRIPTIONS_FIXED, docx_file)
    parsed = parse_docx_lines(docx_path)

    # parsed already returns dict of {page_key: [line1, line2, ...]}
    # but if it's returning strings, split them
    fixed = {}
    for page_key, content in parsed.items():
        if isinstance(content, str):
            # Split string into lines
            lines = [l.strip() for l in content.split('\n') if l.strip()]
        else:
            # Already a list
            lines = [l.strip() for l in content if l.strip()]
        fixed[page_key] = lines

    docx_splits[source_folder] = fixed
    print(f"\n=== {source_folder} ===")
    for page_key, lines in fixed.items():
        print(f"  {page_key}: {len(lines)} lines")
        for i, line in enumerate(lines[:5]):  # preview first 5
            print(f"    [{i:03d}] {line}")

output_path = os.path.join(MANUAL_REVIEW_PATH, "docx_lines.json")
with open(output_path, 'w', encoding='utf-8') as f:
    json.dump(docx_splits, f, ensure_ascii=False, indent=2)
```

```
print(f"\nSaved to: {output_path}")
```

```
=== Guardiola_-_Tratado_nobleza ===
  p12_full: 25 lines
    [000] POR quanto por parte de
    [001] vos fray Ivan Benito Guar
    [002] diola monje professo de
    [003] la orden de S. Benito, nos
    [004] fue fecha relacion q vos
  p13_full: 27 lines
    [000] dia de la fecha desta nuestra cedula podais
    [001] imprimir el dicho libro que de suso se ha-
    [002] ze mencion por el original que en el nuestro
    [003] Consejo se vio que va rubricado y firmado
    [004] al fin de Pedro del Marmol nuestro escri-
  p14_full: 27 lines
    [000] caer e incurrir en las penas contenidas en
    [001] la dicha pragmatica e leyes de nuestros
    [002] Reynos. Y mandamos que durante el di-
    [003] cho tiempo persona alguna sin vuestra li-
    [004] cencia no lo pueda imprimir ni vender so

=== PORCONES.23.5_-_1628 ===
  p1_full: 17 lines
    [000] POR
    [001] DOÑA CATALINA DE
    [002] Laçarraga y Zarate, viuda de Marco
    [003] Antonio de Caycedo, y el señor
    [004] Fiscal.
  p2_left: 33 lines
    [000] ca de Caycedo su hija, donzella noble y honesta, a
    [001] que concedielle en su voluntad, y para esto, y violar su
    [002] castidad, aver quebrantado su casa, siendo de muy luci-
    [003] da nobleza y calidad, entrando en ella denoche a des-
    [004] hora con llaves falsas, y otras vezes por las ventanas, y
  p2_right: 35 lines
    [000] su Magestad) no ha querellado, y la hembra (que es la
    [001] doña Francisca casada oy con el matador) no le pide,
    [002] han de prevalecer a doña Catalina, q acusa por ser ma-
    [003] yor parte.
    [004] Pero este discurso repugna a la jurisprudencia, y a la
  p3_left: 35 lines
    [000] para preferir el mas conjunto, I. 3. §. fi tamen, ff. de libe-
```

```
    [001] r o homine exhib. ibi: Er ex coniunctione, I. 2. §. si mulier,
    [002] ff. de colls. detegen. ibi: Cuius magis interest.
    [003] Por estas razones y otras lo assientan por indubita-
    [004] ble, post Bart. & Bald. Nicol. Boer. decis. 233. a num. 2.
  p3_right: 35 lines
    [000] admitio concurso de acosadores de la muerte con la viu-
    [001] da del muerto, eo magis fiendo persona calificada, y tal
    [002] lexos de colusion, como lo muestra la diligencia, y per-
    [003] severancia con que ha seguido, y sigue este pleyto: y la
    [004] colusion esta en la hija casada oy con el matador de su
  p4_left: 35 lines
    [000] Bosi. tit. de pace, vers. Quando vxor occiditur a viro,
    [001] Iulius Clar. q. 58. num. 31. vers. contingit etiam. Bona-
    [002] cosa in suis communibus, vers. pax vxoricidae, lib. I. Pe-
    [003] trus Cabal. in. resolut. in resolut. crimina. casu 58. an. 10.
    [004] Et tandem, los delitos sobre que es este pleyto son

=== PORCONES.228.38_-_1646 ===
  p1_full: 23 lines
    [000] POR
    [001] ANTONIO GOMEZ
    [002] Oimentel, mercader, padre de
    [003] Iacinto Gomez Pi-
    [004] mentel.
  p2_full: 35 lines
    [000] ticia de esta parte en ajuitar con certeza, y claridad
    [001] el hecho verdadero de este negocio, que consiste
    [002] en que Iacinto Gomez Pimentel fue preso el mes
    [003] de Agosto del año passado de 1643. por querella,
    [004] que de él dio doña Leonor Rauasco, por dezirla
  p3_full: 35 lines
    [000] der seis mil ducados, que auian tocado a su hijo Ia-
    [001] cinto Gomez por su legitima materna: y para que
    [002] dello constasse pidio compulsorio, para que Anto
    [003] nio Cadenas Escriuano de Prouincia, le diesse vn
    [004] testimonio de la particion, que se auia hecho de
  p4_full: 35 lines
    [000] cobraren, se ayan de partir entre todos.
    [001] 10 Antonio Gomez Oimentel, respondió, que
    [002] se auia de declarar no auer lugar la pretensió de do
    [003] ña Leonor Rauasco, porque el tenia pagado a
```

```
    [004] su hijo todo lo que montaua su legitima materna,
  p5_full: 35 lines
    [000] 14 Otra carta de pago otorgada ante el dicho
    [001] Escriuano, en 19. de Enero de 1640. por Iacinto
    [002] Gomez, en que confiessa auer recibido de su padre 9?
    [003] 333. reales, que le tocan, como a vno de tres here-
    [004] deros de doña Beatriz de Silua su madre de su par-

=== PORCONES.748.6_-_1650 ===
  p1_full: 26 lines
    [000] DEUS IN ADIUTORIUM MEUM INTENDE
    [001] POR
    [002] Don Felipe Leizado
    [003] Caballero del Orden Santiago, Ca-
    [004] pitan de Caballos del Ejército
  p2_full: 34 lines
    [000] Artículo primero
    [001] 3. Aunque es regular en estos delitos venéreos la
    [002] prueba por presunciones, ex responso Pontifi-
    [003] cis in cap. literis 12. de presumpt. … incap. Proterea de
    [004] testib us cum notatis ibidem per Canonitas, …Bart in
  p3_full: 35 lines
    [000] platillos si no nos gobernamos con prudencia. Jacome y,
    [001] el cocinero hablan siempre en su lengua, con parientes
    [002] de Matias; y plega a Dios, que una vez u otra no de-
    [003] mos en la trampa. Bien sé que estas mis palabras se echa-
    [004] rán al aire porque vos joya mía no queréis hacer ca-
  p4_full: 11 lines
    [000] tener aquí fuese por falta de voluntad, o bien por mi
    [001] seria de daros el alimento como merecéis, porque si esto
    [002] fuese respecto a vuestra persona, juzgariades la ver-
    [003] dad, pero si después considerasedes, que pasando todo
    [004] por manos de criados, hiciésemos como algunos años

=== Covarrubias_-_Tesoro_lengua ===
  p7_full: 18 lines
    [000] CENSURA DE
    [001] PEDRO DE VALENCIA CORONISTA
    [002] General de el Rey nuestro
    [003] Señor.
    [004] POr mandado del Consejo Supremo de Castilla, he visto el libro in-
```

```
  p8_full: 48 lines
    [000] CARTA DE EL
    [001] LICENCIADO DON BALTASAR
    [002] SEBASTIAN NAVARRO DE ARROYTA, COLEGIAL DEL
    [003] Colegio de San Salvador de Oviedo, el mayor de Salamanca, Sacristan
    [004] mayor, y Canonigo de la santa Catedral Iglesia
  p9_full: 59 lines
    [000] Conocimiento de las cosas depende del de los vocablos, el que ayuda a esta, ayuda mucho a todas
    [001] las ciencias: y assi los grandes Filosofos antiguos trabajaron mucho en esta parte, como fue Platon
    [002] en su Dialogo Cratilo, que oy en dia leemos, a cuya imitacion los Latinos trataron esta facultad,
    [003] aunque con mayores ayudas que los Griegos, por tener mas conocidos los manantiales de su lengua-
    [004] ge, el qual casi todo es Griego, aunque Marco Vatron en los libros qu eescrivio en lengua Latina

=== Buendia_-_Instruccion ===
  p2_full: 22 lines
    [000] Al
    [001] INFINITAMENTE AMABLE
    [002] NIÑO JESUS.
    [003] A Vos, Dulcissimo Niño
    [004] JESUS, que no solo os
  p3_full: 55 lines
    [000] guro disseño de su edad: la Reli-
    [001] gion para con Dios en la devota
    [002] assistencia a los Templos; la piedad
    [003] con los Padres en la obediencia
    [004] mas rendida; y la modestia, y de-
  p4_full: 29 lines
    [000] crianza de la niñez. Assi sea,
    [001] Divinissimo Niño, por vuestra
    [002] gracia, assi sea, a vuestra ma-
    [003] yor gloria. Amen.
    [004] CENSURA DEL R. P. ANTONIO CO-

Saved to: /kaggle/working/manual_review/docx_lines.json
```

In [ ]:

```python
import json
import os

SURYA_LINES_PATH = "surya_lines"
DOCX_LINES_PATH = "docx_lines.json"
MAPPING_OUTPUT = "image_text_mapping.json"
```

```python
# Page key normalization — folder name to docx key
# e.g. "p2_right" folder -> "p2_right" in docx_lines
# e.g. "p3_full" folder -> "p3_full" in docx_lines

with open(DOCX_LINES_PATH, 'r', encoding='utf-8') as f:
    docx_lines = json.load(f)

mapping = {}

for source_folder in sorted(os.listdir(SURYA_LINES_PATH)):
    if source_folder.startswith('.'):
        continue
    source_path = os.path.join(SURYA_LINES_PATH, source_folder)
    if not os.path.isdir(source_path):
        continue

    for page_folder in sorted(os.listdir(source_path)):
        if page_folder.startswith('.'):
            continue
        page_path = os.path.join(source_path, page_folder)
        if not os.path.isdir(page_path):
            continue

        images = sorted([
            os.path.join(page_path, f)
            for f in os.listdir(page_path)
            if f.endswith('.png') and not f.startswith('.')
        ])

        docx_text_lines = docx_lines.get(source_folder, {}).get(page_folder, [])

        # Pair one to one up to min length
        pairs = []
        for img, text in zip(images, docx_text_lines):
            pairs.append({
                "image": img,
                "text": text,
                "source": source_folder,
                "page": page_folder
            })
```

```
        full_key = f"{source_folder}/{page_folder}"
        mapping[full_key] = {
            "n_images": len(images),
            "n_texts": len(docx_text_lines),
            "n_pairs": len(pairs),
            "pairs": pairs
        }
        print(f"{full_key}: {len(images)} images | {len(docx_text_lines)} texts | {len(pairs)} pairs")

with open(MAPPING_OUTPUT, 'w', encoding='utf-8') as f:
    json.dump(mapping, f, ensure_ascii=False, indent=2)

print(f"\nSaved to: {MAPPING_OUTPUT}")
```

```
Buendia_-_Instruccion/p2_full: 20 images | 20 texts | 20 pairs
Buendia_-_Instruccion/p3_full: 56 images | 56 texts | 56 pairs
Buendia_-_Instruccion/p4_full: 29 images | 29 texts | 29 pairs
Covarrubias_-_Tesoro_lengua/p7_full: 17 images | 17 texts | 17 pairs
Covarrubias_-_Tesoro_lengua/p8_full: 49 images | 49 texts | 49 pairs
Covarrubias_-_Tesoro_lengua/p9_full: 60 images | 60 texts | 60 pairs
Guardiola_-_Tratado_nobleza/p12_full: 25 images | 25 texts | 25 pairs
Guardiola_-_Tratado_nobleza/p13_full: 27 images | 27 texts | 27 pairs
Guardiola_-_Tratado_nobleza/p14_full: 27 images | 27 texts | 27 pairs
PORCONES.228.38_-_1646/p1_full: 18 images | 18 texts | 18 pairs
PORCONES.228.38_-_1646/p2_full: 31 images | 31 texts | 31 pairs
PORCONES.228.38_-_1646/p3_full: 34 images | 34 texts | 34 pairs
PORCONES.228.38_-_1646/p4_full: 34 images | 34 texts | 34 pairs
PORCONES.228.38_-_1646/p5_full: 35 images | 35 texts | 35 pairs
PORCONES.23.5_-_1628/p1_full: 16 images | 17 texts | 16 pairs
PORCONES.23.5_-_1628/p2_left: 29 images | 29 texts | 29 pairs
PORCONES.23.5_-_1628/p2_right: 35 images | 35 texts | 35 pairs
PORCONES.23.5_-_1628/p3_left: 35 images | 35 texts | 35 pairs
PORCONES.23.5_-_1628/p3_right: 35 images | 35 texts | 35 pairs
PORCONES.23.5_-_1628/p4_left: 35 images | 35 texts | 35 pairs
PORCONES.748.6_-_1650/p1_full: 26 images | 26 texts | 26 pairs
PORCONES.748.6_-_1650/p2_full: 34 images | 34 texts | 34 pairs
PORCONES.748.6_-_1650/p3_full: 32 images | 32 texts | 32 pairs
PORCONES.748.6_-_1650/p4_full: 11 images | 11 texts | 11 pairs

Saved to: image_text_mapping.json
```

MAE Training Block (Was initially planned to use for Domain adaptation, but integrating MAE + TrOCR didn't give good result)

In [9]:
```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from transformers import ViTModel, ViTConfig
import os
from PIL import Image

MAE_CHECKPOINT = "/kaggle/working/mae_pretrained.pth"
IMG_SIZE, PATCH_SIZE, MASK_RATIO = 384, 16, 0.75
BATCH_SIZE, EPOCHS, LR = 16, 30, 1.5e-4
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
NUM_PATCHES = (IMG_SIZE // PATCH_SIZE) ** 2  # 576
PATCH_DIM = 3 * PATCH_SIZE * PATCH_SIZE      # 768

print(f"Device: {DEVICE} | Num patches: {NUM_PATCHES}")

class UnlabeledPageDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.image_paths = [
            os.path.join(dp, f)
            for dp, dn, filenames in os.walk(root_dir)
            for f in filenames if f.endswith('.jpg')
        ]
        self.transform = transform
        print(f"Total unlabeled images: {len(self.image_paths)}")

    def __len__(self): return len(self.image_paths)

    def __getitem__(self, idx):
        try:
            img = Image.open(self.image_paths[idx]).convert('RGB')
            if self.transform: img = self.transform(img)
            return img
```

```
        except Exception:
            return torch.zeros(3, IMG_SIZE, IMG_SIZE)

transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

class MAEEncoder(nn.Module):
    def __init__(self, img_size=384, patch_size=16, mask_ratio=0.75):
        super().__init__()
        self.patch_size = patch_size
        self.mask_ratio = mask_ratio
        self.patch_dim = 3 * patch_size * patch_size  # 768
        self.num_patches = (img_size // patch_size) ** 2  # 576

        # --- USE TROCR-COMPATIBLE VIT CONFIG ---
        config = ViTConfig(
            image_size=384,
            patch_size=16,
            hidden_size=768,
            num_hidden_layers=12,
            num_attention_heads=12,
            intermediate_size=3072,
        )
        vit = ViTModel(config)

        # Extract components - same naming as TrOCR encoder
        self.embeddings = vit.embeddings      # patch_embed + pos_embed + cls_token
        self.encoder = vit.encoder            # 12 transformer layers
        self.layernorm = vit.layernorm        # final layernorm

        # Lightweight MAE decoder - not saved to final checkpoint
        self.decoder = nn.Sequential(
            nn.Linear(768, 512),
            nn.GELU(),
            nn.Linear(512, self.patch_dim)
        )

        for param in self.parameters():
```

```
            param.requires_grad = True

    def patchify(self, imgs):
        p = self.patch_size
        B, C, H, W = imgs.shape
        x = imgs.reshape(B, C, H//p, p, W//p, p)
        x = x.permute(0, 2, 4, 1, 3, 5).reshape(B, -1, self.patch_dim)
        return x

    def forward(self, imgs):
        B = imgs.shape[0]
        patches = self.patchify(imgs)  # [B, 576, 768] for loss target

        # Get patch embeddings via ViT embeddings module
        # This handles Conv2d patch_embed + pos_embed + cls_token automatically
        x = self.embeddings(imgs)  # [B, 577, 768] — includes cls token

        # Random masking on patch tokens only (skip cls token at index 0)
        patch_tokens = x[:, 1:, :]  # [B, 576, 768]
        N = patch_tokens.shape[1]
        num_keep = int(N * (1 - self.mask_ratio))

        noise = torch.rand(B, N, device=imgs.device)
        ids_shuffle = torch.argsort(noise, dim=1)
        ids_keep = ids_shuffle[:, :num_keep]

        # Keep only visible patches
        visible = torch.gather(
            patch_tokens, dim=1,
            index=ids_keep.unsqueeze(-1).expand(-1, -1, 768)
        )

        # Prepend cls token
        cls_token = x[:, :1, :]
        x_masked = torch.cat([cls_token, visible], dim=1)  # [B, 1+num_keep, 768]

        # Encode
        encoder_out = self.encoder(x_masked)
        x_enc = encoder_out.last_hidden_state
        x_enc = self.layernorm(x_enc)
```

```
        # Decode visible patches only (skip cls)
        x_visible = x_enc[:, 1:, :]
        reconstructed = self.decoder(x_visible)  # [B, num_keep, patch_dim]

        # Loss on visible patches
        target = torch.gather(
            patches, dim=1,
            index=ids_keep.unsqueeze(-1).expand(-1, -1, self.patch_dim)
        )
        loss = nn.functional.mse_loss(reconstructed, target.detach())
        return loss, x_enc
```

```
Device: cuda | Num patches: 576
```

In [10]:

```
UNLABELED_PATH = "/kaggle/input/datasets/indrasn0wal/maetaskocr/kaggle/working/unlabeled_pages"
model = MAEEncoder().to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LR, weight_decay=0.05)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
dataloader = DataLoader(
    UnlabeledPageDataset(UNLABELED_PATH, transform),
    batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True
)

print(f"Training MAE | {EPOCHS} epochs | batch={BATCH_SIZE} | lr={LR}")
```

```
Total unlabeled images: 1373
Training MAE | 30 epochs | batch=16 | lr=0.00015
```

In [11]:

```
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    for batch in dataloader:
        batch = batch.to(DEVICE)
        optimizer.zero_grad()
        loss, _ = model(batch)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        total_loss += loss.item()
    scheduler.step()
    avg_loss = total_loss / len(dataloader)
```

```
    print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f}")
    if (epoch + 1) % 10 == 0:
        torch.save(model.state_dict(), MAE_CHECKPOINT)
        print(f"Checkpoint saved at epoch {epoch+1}")

torch.save(model.state_dict(), MAE_CHECKPOINT)
print(f"MAE pretraining complete. Size: {os.path.getsize(MAE_CHECKPOINT)/1024/1024:.1f} MB")
```

```
Epoch 1/30 | Loss: 0.0891
Epoch 2/30 | Loss: 0.0556
Epoch 3/30 | Loss: 0.0422
Epoch 4/30 | Loss: 0.0360
Epoch 5/30 | Loss: 0.0318
Epoch 6/30 | Loss: 0.0247
Epoch 7/30 | Loss: 0.0200
Epoch 8/30 | Loss: 0.0168
Epoch 9/30 | Loss: 0.0145
Epoch 10/30 | Loss: 0.0125
Checkpoint saved at epoch 10
Epoch 11/30 | Loss: 0.0127
Epoch 12/30 | Loss: 0.0105
Epoch 13/30 | Loss: 0.0094
Epoch 14/30 | Loss: 0.0087
Epoch 15/30 | Loss: 0.0096
Epoch 16/30 | Loss: 0.0079
Epoch 17/30 | Loss: 0.0072
Epoch 18/30 | Loss: 0.0069
Epoch 19/30 | Loss: 0.0065
Epoch 20/30 | Loss: 0.0063
Checkpoint saved at epoch 20
Epoch 21/30 | Loss: 0.0060
Epoch 22/30 | Loss: 0.0059
Epoch 23/30 | Loss: 0.0057
Epoch 24/30 | Loss: 0.0056
Epoch 25/30 | Loss: 0.0055
Epoch 26/30 | Loss: 0.0054
Epoch 27/30 | Loss: 0.0054
Epoch 28/30 | Loss: 0.0054
Epoch 29/30 | Loss: 0.0054
Epoch 30/30 | Loss: 0.0053
Checkpoint saved at epoch 30
MAE pretraining complete. Size: 331.5 MB
```

In [13]:

```python
import shutil
shutil.copy(MAE_CHECKPOINT, "/kaggle/working/mae_pretrained_v2_384.pth")
print("Backup saved")
```

```
Backup saved
```

Generating Synthetic text line images (TRDG)

In [1]:
```
import json
import random

MAPPING_PATH = "/kaggle/input/datasets/indrasn0wal/humanai-mapping-data-json/mapping data/image_text_mapping.json"
TEST_LINES_PATH = "/kaggle/working/test_lines.json"
TRAIN_LINES_PATH = "/kaggle/working/train_lines.json"

with open(MAPPING_PATH, 'r', encoding='utf-8') as f:
    mapping = json.load(f)

# Collect all pairs across all pages
all_pairs = []
for page_key, page_data in mapping.items():
    for pair in page_data['pairs']:
        all_pairs.append(pair)

print(f"Total pairs: {len(all_pairs)}")

# Shuffle and split
random.seed(42)
random.shuffle(all_pairs)

test_pairs = all_pairs[:200]
train_pairs = all_pairs[200:]

print(f"Test pairs (real scanned images): {len(test_pairs)}")
print(f"Train pairs (text only → TRDG): {len(train_pairs)}")

# Save
with open(TEST_LINES_PATH, 'w', encoding='utf-8') as f:
    json.dump(test_pairs, f, ensure_ascii=False, indent=2)

with open(TRAIN_LINES_PATH, 'w', encoding='utf-8') as f:
    json.dump(train_pairs, f, ensure_ascii=False, indent=2)
```

```
print(f"\nSaved test: {TEST_LINES_PATH}")
print(f"Saved train: {TRAIN_LINES_PATH}")
```

```
Total pairs: 750
Test pairs (real scanned images): 200
Train pairs (text only → TRDG): 550

Saved test: /kaggle/working/test_lines.json
Saved train: /kaggle/working/train_lines.json
```

In [5]:

```
!pip install "pip<24.1" -q
!pip install trdg -q
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 33.3 MB/s eta 0:00:00a 0:00:01
  Preparing metadata (setup.py) ... done
  Preparing metadata (setup.py) ... done
WARNING: The candidate selected for download or install is a yanked version: 'arabic-reshaper' candidate (version 2.
1.3 at https://files.pythonhosted.org/packages/47/27/7b9b824f5342d8ee180027333f2e15842ea36f5bc2d3d24a4e6bb31fb596/ar
abic_reshaper-2.1.3-py3-none-any.whl (from https://pypi.org/simple/arabic-reshaper/))
Reason for being yanked: Doesn't work with Python 2
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 98.6/98.6 MB 8.8 MB/s eta 0:00:00:00:0100:01
  Building wheel for diffimg (setup.py) ... done
  Building wheel for wikipedia (setup.py) ... done
```

In [6]:

```
import json
import os
import random
import numpy as np
from PIL import Image, ImageDraw, ImageFont, ImageFilter, ImageEnhance

TRAIN_LINES_PATH = "/kaggle/working/train_lines.json"
SYNTHETIC_DIR = "/kaggle/working/synthetic_lines"
SYNTHETIC_PAIRS_PATH = "/kaggle/working/synthetic_pairs.json"
os.makedirs(SYNTHETIC_DIR, exist_ok=True)

FONTS = [
    ("/usr/share/fonts/opentype/urw-base35/NimbusRoman-Regular.otf", 32),
    ("/usr/share/fonts/opentype/urw-base35/P052-Italic.otf", 30),
    ("/usr/share/fonts/opentype/urw-base35/C059-Roman.otf", 34),
]
SIZE_VARIATIONS = [-2, 0, 2]
```

```python
def degrade_image(img):
    img = img.convert("RGB")
    arr = np.array(img, dtype=np.float32)
    white_mask = (arr[:,:,0] > 200) & (arr[:,:,1] > 200) & (arr[:,:,2] > 200)
    paper_color = [
        random.randint(225, 245),
        random.randint(210, 230),
        random.randint(175, 205),
    ]
    arr[white_mask] = paper_color
    noise = np.random.normal(0, random.uniform(3, 12), arr.shape)
    arr = np.clip(arr + noise, 0, 255).astype(np.uint8)
    img = Image.fromarray(arr)
    if random.random() > 0.4:
        img = img.filter(ImageFilter.GaussianBlur(radius=random.uniform(0.3, 0.8)))
    img = ImageEnhance.Contrast(img).enhance(random.uniform(0.7, 0.95))
    img = ImageEnhance.Brightness(img).enhance(random.uniform(0.85, 1.1))
    return img

def render_text_line(text, font_path, font_size, padding=10):
    try:
        font = ImageFont.truetype(font_path, font_size)
    except:
        font = ImageFont.load_default()
    dummy = Image.new("RGB", (1, 1))
    draw = ImageDraw.Draw(dummy)
    bbox = draw.textbbox((0, 0), text, font=font)
    w = bbox[2] - bbox[0] + padding * 2
    h = bbox[3] - bbox[1] + padding * 2
    img = Image.new("RGB", (max(w, 50), max(h, 20)), (255, 255, 255))
    draw = ImageDraw.Draw(img)
    draw.text((padding, padding), text, font=font, fill=(0, 0, 0))
    return img

with open(TRAIN_LINES_PATH, 'r', encoding='utf-8') as f:
    train_lines = json.load(f)

texts = list(set([p['text'] for p in train_lines if p['text'].strip()]))
print(f"Unique text lines: {len(texts)}")
```

```python
synthetic_pairs = []
img_counter = 0

for font_path, font_size in FONTS:
    for size_var in SIZE_VARIATIONS:
        for text in texts:
            try:
                img = render_text_line(text, font_path, font_size + size_var)
                img = degrade_image(img)
                save_path = os.path.join(SYNTHETIC_DIR, f"syn_{img_counter:05d}.png")
                img.save(save_path, "PNG")
                synthetic_pairs.append({
                    "image": save_path,
                    "text": text,
                    "font": os.path.basename(font_path)
                })
                img_counter += 1
            except Exception as e:
                continue

        print(f"Font: {os.path.basename(font_path)} | size_var={size_var} | Total: {img_counter}")

with open(SYNTHETIC_PAIRS_PATH, 'w', encoding='utf-8') as f:
    json.dump(synthetic_pairs, f, ensure_ascii=False, indent=2)

print(f"\nTotal synthetic pairs: {len(synthetic_pairs)}")
print(f"Saved to: {SYNTHETIC_PAIRS_PATH}")
```

```
Unique text lines: 548
Font: NimbusRoman-Regular.otf | size_var=-2 | Total: 548
Font: NimbusRoman-Regular.otf | size_var=0 | Total: 1096
Font: NimbusRoman-Regular.otf | size_var=2 | Total: 1644
Font: P052-Italic.otf | size_var=-2 | Total: 2192
Font: P052-Italic.otf | size_var=0 | Total: 2740
Font: P052-Italic.otf | size_var=2 | Total: 3288
Font: C059-Roman.otf | size_var=-2 | Total: 3836
Font: C059-Roman.otf | size_var=0 | Total: 4384
Font: C059-Roman.otf | size_var=2 | Total: 4932

Total synthetic pairs: 4932
Saved to: /kaggle/working/synthetic_pairs.json
```

TrOCR fine-tune

1. First Fine-tuned on the 4932 synthetic text image pair
2. Then The 548 text lines used to create the synthetic image pair, were used to fine-tune it (i.e the origina text image scans)

In [5]:
```
!pip install evaluate
```

```
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Requirement already satisfied: datasets>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from evaluate) (4.0.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from evaluate) (2.0.2)
Requirement already satisfied: dill in /usr/local/lib/python3.12/dist-packages (from evaluate) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from evaluate) (2.3.3)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from evaluate) (2.32.4)
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/dist-packages (from evaluate) (4.67.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from evaluate) (3.6.0)
Requirement already satisfied: multiprocess in /usr/local/lib/python3.12/dist-packages (from evaluate) (0.70.16)
Requirement already satisfied: fsspec>=2021.05.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]>=202
1.05.0->evaluate) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from evaluate) (1.
4.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from evaluate) (25.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets>=2.0.0->evaluate)
(3.20.3)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets>=2.0.0->eva
luate) (18.1.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets>=2.0.0->evaluat
e) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[ht
tp]>=2021.05.0->evaluate) (3.13.3)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub
>=0.7.0->evaluate) (1.2.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.
7.0->evaluate) (0.28.1)
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.7.0->
evaluate) (1.5.4)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.7.0->e
valuate) (0.21.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface
-hub>=0.7.0->evaluate) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=
2.19.0->evaluate) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->evalu
ate) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0-
>evaluate) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0-
>evaluate) (2026.1.4)
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->evalu
ate) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->evaluate) (202
5.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->evaluate) (20
25.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.
0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=
4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.
0.0a1->fsspec[http]>=2021.05.0->evaluate) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a
0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (6.7.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=
4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (1.22.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-
hub>=0.7.0->evaluate) (4.12.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->hugg
ingface-hub>=0.7.0->evaluate) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=
0.23.0->huggingface-hub>=0.7.0->evaluate) (0.16.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pan
das->evaluate) (1.17.0)
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim->huggingface
-hub>=0.7.0->evaluate) (8.3.1)
Downloading evaluate-0.4.6-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 3.0 MB/s eta 0:00:00
Installing collected packages: evaluate
Successfully installed evaluate-0.4.6
```

In [4]:

```python
import json
import os
import gc
import torch
import evaluate
from torch.utils.data import Dataset, DataLoader
```

```python
from torch.optim import AdamW
from transformers import TrOCRProcessor, VisionEncoderDecoderModel, GenerationConfig
from sklearn.model_selection import train_test_split
from PIL import Image
from tqdm import tqdm

# --- CONFIG ---
MODEL_NAME = "microsoft/trocr-base-printed"
SYNTHETIC_PAIRS_PATH = "/kaggle/input/datasets/indrasn0wal/json-mapping/synthetic_pairs.json"
BEST_MODEL_PATH = "/kaggle/working/trocr_synthetic"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
EPOCHS = 5
BATCH_SIZE = 1
ACCUMULATION_STEPS = 8
LR = 2e-5
BASE_PREFIX = "/kaggle/input/datasets/indrasn0wal/synthetic-dataset"

os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"

# --- DATASET ---
class OCRDataset(Dataset):
    def __init__(self, pairs, processor):
        self.pairs = pairs
        self.processor = processor

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        pair = self.pairs[idx]
        try:
            img = Image.open(BASE_PREFIX + pair['image']).convert("RGB")
        except:
            img = Image.new('RGB', (384, 48), (255, 255, 255))

        pixel_values = self.processor(img, return_tensors="pt").pixel_values.squeeze(0)
        labels = self.processor.tokenizer(
            pair['text'],
            padding="max_length",
            max_length=64,
            truncation=True,
```

```
            return_tensors="pt"
        ).input_ids.squeeze(0)
        labels[labels == self.processor.tokenizer.pad_token_id] = -100
        return pixel_values, labels

# --- LOAD DATA ---
with open(SYNTHETIC_PAIRS_PATH, 'r', encoding='utf-8') as f:
    all_pairs = json.load(f)

train_pairs, val_pairs = train_test_split(all_pairs, test_size=0.1, random_state=42)
print(f"Train: {len(train_pairs)} | Val: {len(val_pairs)}")

# --- LOAD MODEL ---
processor = TrOCRProcessor.from_pretrained(MODEL_NAME)
model = VisionEncoderDecoderModel.from_pretrained(MODEL_NAME)

# --- MODEL CONFIG ---
model.config.decoder_start_token_id = processor.tokenizer.cls_token_id
model.config.pad_token_id = processor.tokenizer.pad_token_id
model.config.eos_token_id = processor.tokenizer.sep_token_id
model.config.vocab_size = model.config.decoder.vocab_size

# --- GENERATION CONFIG ---
model.generation_config = GenerationConfig(
    decoder_start_token_id=processor.tokenizer.cls_token_id,
    pad_token_id=processor.tokenizer.pad_token_id,
    eos_token_id=processor.tokenizer.sep_token_id,
    max_new_tokens=32,
    num_beams=1,
)

model.to(DEVICE)
print("No MAE injection — vanilla TrOCR")

# --- DATALOADERS ---
train_loader = DataLoader(
    OCRDataset(train_pairs, processor),
    batch_size=BATCH_SIZE, shuffle=True, num_workers=0
)
val_loader = DataLoader(
    OCRDataset(val_pairs, processor),
```

```
    batch_size=1, shuffle=False, num_workers=0
)

# --- OPTIMIZER ---
optimizer = AdamW(model.parameters(), lr=LR)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=EPOCHS, eta_min=1e-6
)
cer_metric = evaluate.load("cer")
best_cer = float('inf')

# --- TRAINING LOOP ---
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    optimizer.zero_grad()

    for step, (pixel_values, labels) in enumerate(
        tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}")
    ):
        pixel_values = pixel_values.to(DEVICE)
        labels = labels.to(DEVICE)

        outputs = model(pixel_values=pixel_values, labels=labels)
        loss = outputs.loss / ACCUMULATION_STEPS
        loss.backward()
        total_loss += loss.item() * ACCUMULATION_STEPS

        if (step + 1) % ACCUMULATION_STEPS == 0:
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            optimizer.zero_grad()

    scheduler.step()
    avg_loss = total_loss / len(train_loader)

    # Validation
    model.eval()
    preds, refs = [], []
    with torch.no_grad():
        for pixel_values, labels in tqdm(val_loader, desc="[Validating]"):
```

```
            pixel_values = pixel_values.to(DEVICE)
            generated = model.generate(
                pixel_values,
                generation_config=model.generation_config
            )
            pred = processor.batch_decode(generated, skip_special_tokens=True)
            ref_ids = labels.squeeze(0)
            ref_ids[ref_ids == -100] = processor.tokenizer.pad_token_id
            ref = processor.batch_decode(ref_ids.unsqueeze(0), skip_special_tokens=True)
            preds.extend(pred)
            refs.extend(ref)

    # Lowercase normalization
    cer = cer_metric.compute(
        predictions=[p.lower() for p in preds],
        references=[r.lower() for r in refs]
    )
    print(f"\n✨ Epoch {epoch+1} | Loss: {avg_loss:.4f} | CER: {cer:.4f}")

    if cer < best_cer:
        best_cer = cer
        model.save_pretrained(BEST_MODEL_PATH)
        processor.save_pretrained(BEST_MODEL_PATH)
        print(f"🔥 New Best CER: {best_cer:.4f} saved to {BEST_MODEL_PATH}")

    gc.collect()
    torch.cuda.empty_cache()

print(f"\nStage 1 complete. Best CER: {best_cer:.4f}")
```

```
Train: 4438 | Val: 494
preprocessor_config.json:   0%|          | 0.00/224 [00:00<?, ?B/s]
The image processor of type `ViTImageProcessor` is now loaded as a fast processor by default, even if the model chec
kpoint was saved with a slow processor. This is a breaking change and may produce slightly different outputs. To con
tinue using the slow processor, instantiate this class with `use_fast=False`.
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
and faster downloads.
config.json: 0.00B [00:00, ?B/s]
tokenizer_config.json: 0.00B [00:00, ?B/s]
special_tokens_map.json:   0%|          | 0.00/772 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
```

```
merges.txt: 0.00B [00:00, ?B/s]
model.safetensors:   0%|          | 0.00/1.33G [00:00<?, ?B/s]
Loading weights:   0%|          | 0/478 [00:00<?, ?it/s]
VisionEncoderDecoderModel LOAD REPORT from: microsoft/trocr-base-printed
Key                          | Status  |
-----------------------------+---------+-
encoder.pooler.dense.weight | MISSING |
encoder.pooler.dense.bias   | MISSING |

Notes:
- MISSING       :those params were newly initialized because missing from the checkpoint. Consider training on your
downstream task.
generation_config.json:   0%|          | 0.00/190 [00:00<?, ?B/s]
No MAE injection — vanilla TrOCR
Downloading builder script: 0.00B [00:00, ?B/s]
Epoch 1/5: 100%|██████████| 4438/4438 [16:13<00:00,  4.56it/s]
[Validating]: 100%|██████████| 494/494 [02:22<00:00,  3.46it/s]
✨ Epoch 1 | Loss: 0.3270 | CER: 0.0433
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.0433 saved to /kaggle/working/trocr_synthetic
Epoch 2/5: 100%|██████████| 4438/4438 [15:57<00:00,  4.63it/s]
[Validating]: 100%|██████████| 494/494 [02:17<00:00,  3.58it/s]
✨ Epoch 2 | Loss: 0.0981 | CER: 0.0369
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.0369 saved to /kaggle/working/trocr_synthetic
Epoch 3/5: 100%|██████████| 4438/4438 [15:57<00:00,  4.64it/s]
[Validating]: 100%|██████████| 494/494 [02:17<00:00,  3.59it/s]
✨ Epoch 3 | Loss: 0.0305 | CER: 0.0285
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.0285 saved to /kaggle/working/trocr_synthetic
Epoch 4/5: 100%|██████████| 4438/4438 [15:58<00:00,  4.63it/s]
[Validating]: 100%|██████████| 494/494 [02:18<00:00,  3.58it/s]
✨ Epoch 4 | Loss: 0.0063 | CER: 0.0244
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.0244 saved to /kaggle/working/trocr_synthetic
Epoch 5/5: 100%|██████████| 4438/4438 [15:57<00:00,  4.63it/s]
[Validating]: 100%|██████████| 494/494 [02:17<00:00,  3.59it/s]
```

```
✨ Epoch 5 | Loss: 0.0020 | CER: 0.0244

Stage 1 complete. Best CER: 0.0244
```

In [5]:
```python
import shutil

# Copy to output directory for persistence
shutil.copytree(
    "/kaggle/working/trocr_synthetic",
    "/kaggle/working/trocr_synthetic_backup",
    dirs_exist_ok=True
)
print("Model backed up")
```

```
Model backed up
```

On Real data

In [12]:
```python
!pip install jiwer
```

```
Collecting jiwer
  Downloading jiwer-4.0.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: click>=8.1.8 in /usr/local/lib/python3.12/dist-packages (from jiwer) (8.3.1)
Collecting rapidfuzz>=3.9.7 (from jiwer)
  Downloading rapidfuzz-3.14.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (12 kB)
Downloading jiwer-4.0.0-py3-none-any.whl (23 kB)
Downloading rapidfuzz-3.14.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 35.3 MB/s eta 0:00:00a 0:00:01
Installing collected packages: rapidfuzz, jiwer
Successfully installed jiwer-4.0.0 rapidfuzz-3.14.3
```

In [13]:
```python
import json
import os
import gc
import torch
import evaluate
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
from transformers import TrOCRProcessor, VisionEncoderDecoderModel, GenerationConfig
from sklearn.model_selection import train_test_split
from PIL import Image
```

```
from tqdm import tqdm

# --- CONFIG ---
# 1. Point to your saved synthetic model instead of the HuggingFace base model
#Uploaded the model in kaggle inputs.
SYNTHETIC_MODEL_PATH = "/kaggle/input/models/indrasn0wal/vanila-synthetic-trocr/pytorch/default/1/kaggle/working/tr

REAL_PAIRS_PATH = "/kaggle/input/datasets/indrasn0wal/json-mapping/train_lines.json"

# 2. Update the save path to the new real checkpoint folder
BEST_MODEL_PATH = "/kaggle/working/trocr_real_checkpoint"

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
EPOCHS = 10
BATCH_SIZE = 1
ACCUMULATION_STEPS = 8
LR = 2e-5
BASE_PREFIX = "/kaggle/input/datasets/indrasn0wal/humanai-mapping-data-json/mapping data/"

os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"

# --- DATASET ---
class OCRDataset(Dataset):
    def __init__(self, pairs, processor):
        self.pairs = pairs
        self.processor = processor

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        pair = self.pairs[idx]
        try:
            img = Image.open(BASE_PREFIX + pair['image']).convert("RGB")
        except:
            img = Image.new('RGB', (384, 48), (255, 255, 255))

        pixel_values = self.processor(img, return_tensors="pt").pixel_values.squeeze(0)
        labels = self.processor.tokenizer(
            pair['text'],
            padding="max_length",
```

```
            max_length=64,
            truncation=True,
            return_tensors="pt"
        ).input_ids.squeeze(0)
        labels[labels == self.processor.tokenizer.pad_token_id] = -100
        return pixel_values, labels

# --- LOAD DATA ---
with open(REAL_PAIRS_PATH, 'r', encoding='utf-8') as f:
    all_pairs = json.load(f)

train_pairs, val_pairs = train_test_split(all_pairs, test_size=0.1, random_state=42)
print(f"Train: {len(train_pairs)} | Val: {len(val_pairs)}")

# --- LOAD MODEL (Modified to load your synthetic weights) ---
processor = TrOCRProcessor.from_pretrained(SYNTHETIC_MODEL_PATH)
model = VisionEncoderDecoderModel.from_pretrained(SYNTHETIC_MODEL_PATH)

# --- MODEL CONFIG ---
model.config.decoder_start_token_id = processor.tokenizer.cls_token_id
model.config.pad_token_id = processor.tokenizer.pad_token_id
model.config.eos_token_id = processor.tokenizer.sep_token_id
model.config.vocab_size = model.config.decoder.vocab_size

# --- GENERATION CONFIG ---
model.generation_config = GenerationConfig(
    decoder_start_token_id=processor.tokenizer.cls_token_id,
    pad_token_id=processor.tokenizer.pad_token_id,
    eos_token_id=processor.tokenizer.sep_token_id,
    max_new_tokens=32,
    num_beams=1,
)

model.to(DEVICE)
print(f"Loaded Synthetic Checkpoint from: {SYNTHETIC_MODEL_PATH}")
print(f"Fine-tuning on REAL Data. Will save best model to: {BEST_MODEL_PATH}")

# --- DATALOADERS ---
train_loader = DataLoader(
    OCRDataset(train_pairs, processor),
    batch_size=BATCH_SIZE, shuffle=True, num_workers=0
```

```
)
val_loader = DataLoader(
    OCRDataset(val_pairs, processor),
    batch_size=1, shuffle=False, num_workers=0
)

# --- OPTIMIZER ---
optimizer = AdamW(model.parameters(), lr=LR)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=EPOCHS, eta_min=1e-6
)
cer_metric = evaluate.load("cer")
best_cer = float('inf')

# --- TRAINING LOOP ---
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    optimizer.zero_grad()

    for step, (pixel_values, labels) in enumerate(
        tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}")
    ):
        pixel_values = pixel_values.to(DEVICE)
        labels = labels.to(DEVICE)

        outputs = model(pixel_values=pixel_values, labels=labels)
        loss = outputs.loss / ACCUMULATION_STEPS
        loss.backward()
        total_loss += loss.item() * ACCUMULATION_STEPS

        if (step + 1) % ACCUMULATION_STEPS == 0:
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            optimizer.zero_grad()

    scheduler.step()
    avg_loss = total_loss / len(train_loader)

    # Validation
    model.eval()
```

```
    preds, refs = [], []
    with torch.no_grad():
        for pixel_values, labels in tqdm(val_loader, desc="[Validating]"):
            pixel_values = pixel_values.to(DEVICE)
            generated = model.generate(
                pixel_values,
                generation_config=model.generation_config
            )
            pred = processor.batch_decode(generated, skip_special_tokens=True)
            ref_ids = labels.squeeze(0)
            ref_ids[ref_ids == -100] = processor.tokenizer.pad_token_id
            ref = processor.batch_decode(ref_ids.unsqueeze(0), skip_special_tokens=True)
            preds.extend(pred)
            refs.extend(ref)

    # Lowercase normalization
    cer = cer_metric.compute(
        predictions=[p.lower() for p in preds],
        references=[r.lower() for r in refs]
    )
    print(f"\n✨ Epoch {epoch+1} | Loss: {avg_loss:.4f} | CER: {cer:.4f}")

    if cer < best_cer:
        best_cer = cer
        model.save_pretrained(BEST_MODEL_PATH)
        processor.save_pretrained(BEST_MODEL_PATH)
        print(f"🔥 New Best CER: {best_cer:.4f} saved to {BEST_MODEL_PATH}")

    gc.collect()
    torch.cuda.empty_cache()

print(f"\nReal Data Fine-tuning complete. Best CER: {best_cer:.4f}")
```

```
Train: 495 | Val: 55
Loading weights:   0%|          | 0/480 [00:00<?, ?it/s]
Loaded Synthetic Checkpoint from: /kaggle/input/models/indrasn0wal/vanila-synthetic-trocr/pytorch/default/1/kaggle/w
orking/trocr_synthetic
Fine-tuning on REAL Data. Will save best model to: /kaggle/working/trocr_real_checkpoint
Epoch 1/10: 100%|██████████| 495/495 [02:18<00:00,  3.56it/s]
[Validating]: 100%|██████████| 55/55 [00:18<00:00,  2.94it/s]
✨ Epoch 1 | Loss: 0.9779 | CER: 0.1571
```

```
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.1571 saved to /kaggle/working/trocr_real_checkpoint
Epoch 2/10: 100%|██████████| 495/495 [02:09<00:00,  3.83it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.16it/s]
✨ Epoch 2 | Loss: 0.4130 | CER: 0.1459
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.1459 saved to /kaggle/working/trocr_real_checkpoint
Epoch 3/10: 100%|██████████| 495/495 [02:09<00:00,  3.82it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.16it/s]
✨ Epoch 3 | Loss: 0.2363 | CER: 0.1622
Epoch 4/10: 100%|██████████| 495/495 [02:09<00:00,  3.83it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.15it/s]
✨ Epoch 4 | Loss: 0.1655 | CER: 0.1361
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.1361 saved to /kaggle/working/trocr_real_checkpoint
Epoch 5/10: 100%|██████████| 495/495 [02:09<00:00,  3.83it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.17it/s]
✨ Epoch 5 | Loss: 0.1197 | CER: 0.1405
Epoch 6/10: 100%|██████████| 495/495 [02:08<00:00,  3.85it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.19it/s]
✨ Epoch 6 | Loss: 0.0860 | CER: 0.1301
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.1301 saved to /kaggle/working/trocr_real_checkpoint
Epoch 7/10: 100%|██████████| 495/495 [02:09<00:00,  3.82it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.18it/s]
✨ Epoch 7 | Loss: 0.0528 | CER: 0.1270
Writing model shards:   0%|          | 0/1 [00:00<?, ?it/s]
🔥 New Best CER: 0.1270 saved to /kaggle/working/trocr_real_checkpoint
Epoch 8/10: 100%|██████████| 495/495 [02:09<00:00,  3.81it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.17it/s]
✨ Epoch 8 | Loss: 0.0327 | CER: 0.1392
Epoch 9/10: 100%|██████████| 495/495 [02:08<00:00,  3.85it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.20it/s]
✨ Epoch 9 | Loss: 0.0265 | CER: 0.1361
Epoch 10/10: 100%|██████████| 495/495 [02:09<00:00,  3.83it/s]
[Validating]: 100%|██████████| 55/55 [00:17<00:00,  3.16it/s]
```

```
✨ Epoch 10 | Loss: 0.0174 | CER: 0.1311

Real Data Fine-tuning complete. Best CER: 0.1270
```

Test Scipt

In [ ]:
```
import json
import torch
import evaluate
from transformers import TrOCRProcessor, VisionEncoderDecoderModel, GenerationConfig
from PIL import Image
from tqdm import tqdm

# --- CONFIG ---
# Uploaded the final fine-tuned model in kaggle inputs. This is the model path 'BEST_MODEL_PATH'
BEST_MODEL_PATH = "/kaggle/input/models/indrasn0wal/vanilla-trocr-real-checkpoint/pytorch/default/1"
TEST_LINES_PATH = "/kaggle/input/datasets/indrasn0wal/new-test-json/accept_test_lines.json"
BASE_IMAGE_PATH = "/kaggle/input/datasets/indrasn0wal/humanai-mapping-data-json/mapping data/"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# --- LOAD MODEL ---
processor = TrOCRProcessor.from_pretrained(BEST_MODEL_PATH)
model = VisionEncoderDecoderModel.from_pretrained(BEST_MODEL_PATH)
model.config.decoder_start_token_id = processor.tokenizer.cls_token_id
model.config.pad_token_id = processor.tokenizer.pad_token_id
model.config.eos_token_id = processor.tokenizer.sep_token_id

model.generation_config = GenerationConfig(
    decoder_start_token_id=processor.tokenizer.cls_token_id,
    pad_token_id=processor.tokenizer.pad_token_id,
    eos_token_id=processor.tokenizer.sep_token_id,
    max_new_tokens=32,
    num_beams=4,  # beam search for final evaluation
)

model.to(DEVICE)
model.eval()

# --- LOAD TEST DATA ---
```

```python
with open(TEST_LINES_PATH, 'r', encoding='utf-8') as f:
    test_pairs = json.load(f)

print(f"Test pairs: {len(test_pairs)}")

# --- EVALUATE ---
cer_metric = evaluate.load("cer")
wer_metric = evaluate.load("wer")
preds, refs, results = [], [], []

with torch.no_grad():
    for pair in tqdm(test_pairs, desc="Evaluating"):
        try:
            full_path = BASE_IMAGE_PATH + pair['image']
            img = Image.open(full_path).convert("RGB")
        except:
            img = Image.new('RGB', (384, 48), (255, 255, 255))

        pixels = processor(img, return_tensors="pt").pixel_values.to(DEVICE)
        generated = model.generate(
            pixels,
            generation_config=model.generation_config
        )
        pred = processor.batch_decode(generated, skip_special_tokens=True)[0]
        ref = pair['text']

        preds.append(pred.lower())
        refs.append(ref.lower())
        results.append({
            "image": pair['image'],
            "predicted": pred,
            "expected": ref,
            "source": pair.get('source', ''),
            "page": pair.get('page', '')
        })

# --- METRICS ---
cer = cer_metric.compute(predictions=preds, references=refs)
wer = wer_metric.compute(predictions=preds, references=refs)

print(f"\n{'='*50}")
```

```
print(f"Final CER: {cer:.4f} ({cer*100:.2f}%)")
print(f"Final WER: {wer:.4f} ({wer*100:.2f}%)")
print(f"{'='*50}")

# Per source
print("\n=== CER per source ===")
from collections import defaultdict
source_preds = defaultdict(list)
source_refs = defaultdict(list)
for r in results:
    source_preds[r['source']].append(r['predicted'].lower())
    source_refs[r['source']].append(r['expected'].lower())

for source in sorted(source_preds.keys()):
    source_cer = cer_metric.compute(
        predictions=source_preds[source],
        references=source_refs[source]
    )
    print(f"  {source}: CER={source_cer:.4f} ({len(source_preds[source])} pairs)")

# Sample predictions
print("\n=== Sample Predictions ===")
for r in results[:10]:
    print(f"Predicted: '{r['predicted']}'")
    print(f"Expected:  '{r['expected']}'")
    print()

# Save
with open("/kaggle/working/test_results.json", 'w', encoding='utf-8') as f:
    json.dump({
        "overall_cer": cer,
        "overall_wer": wer,
        "total_pairs": len(test_pairs),
        "results": results
    }, f, ensure_ascii=False, indent=2)

print(f"Saved to /kaggle/working/test_results.json")
```

```
Loading weights:   0%|          | 0/480 [00:00<?, ?it/s]
Test pairs: 195
Downloading builder script: 0.00B [00:00, ?B/s]
```

```
Downloading builder script: 0.00B [00:00, ?B/s]
Evaluating: 100%|██████████| 195/195 [01:27<00:00,  2.23it/s]
```

```
==================================================
Final CER: 0.1133 (11.33%)
Final WER: 0.2740 (27.40%)
==================================================

=== CER per source ===
  Buendia_-_Instruccion: CER=0.0342 (27 pairs)
  Covarrubias_-_Tesoro_lengua: CER=0.1731 (34 pairs)
  Guardiola_-_Tratado_nobleza: CER=0.0458 (20 pairs)
  PORCONES.228.38_-_1646: CER=0.1008 (43 pairs)
  PORCONES.23.5_-_1628: CER=0.1009 (51 pairs)
  PORCONES.748.6_-_1650: CER=0.1139 (20 pairs)

=== Sample Predictions ===
Predicted: 'yor parte. No.'
Expected:  'yor parte.'

Predicted: 'alomenos concurrir con ella: y de aqui infirió, que pues'
Expected:  'alomenos concurrir con ella: y que de aqui infirio, que pues'

Predicted: 'hallarse sabdita de su marido, y prohibida a acusalla: y'
Expected:  'hallarse subdita de su marido, y prohibida a acusalle: y'

Predicted: 'de 3. ibi. Primos silicet vxor, secundo pater, testio liberi'
Expected:  '& 3. ibi: Primo scilicet uxor, secundo pater, tertio liberi'

Predicted: 'ua, y ratifica. Y ansimismo declara, que tiene otor'
Expected:  'ua, y ratifica. Y ansimismo declara, que tiene otor'

Predicted: '10 Lo mismo que está dicho en la acusació de la muer-'
Expected:  '10 Lo mismo que esta dicho en la acusacion de la muer-'

Predicted: 'quoniam in illa concurrimt maior coniunctiono? afecto'
Expected:  'quoniam in illa concurrunt maior coniunctio & affectio'

Predicted: 'lo que deben estudiar. Por este so-'
Expected:  'lo que deben estudiar. Por este so-'

Predicted: 'cion. Vos, como linguar Maes-'
Expected:  'cion. Vos, como singular Maes-'
```

```
Predicted: 'recerles. Sean tan servorosamen-'
Expected:  'recerles. Sean tan fervorosamen-'

Saved to /kaggle/working/test_results.json
```

LLM Post processing (Which was not much helpful, so we switched to VLM in our inference as it helped in our Drop Cap letter insertion)

In [ ]:

```
import json
import re
import time
import google.generativeai as genai
import evaluate

# --- CONFIG ---
GEMINI_API_KEY = "your-api-key"
TEST_RESULTS_PATH = "/kaggle/working/test_results.json"
LLM_RESULTS_PATH = "/kaggle/working/llm_results.json"

genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("gemini-2.0-flash")

# --- RULE-BASED NORMALIZATION FIRST ---
def rule_based_normalize(text):
    """ Apply known early modern Spanish normalization rules from docx notes."""
    # ç → z
    text = text.replace('ç', 'z').replace('Ç', 'Z')
    # ũ/ũ → un (nasal vowel expansion)
    text = re.sub(r'[ũ]', 'un', text)
    # q̃ → que
    text = re.sub(r'q̃', 'que', text)
    # long-s (ſ) → s
    text = text.replace('ſ', 's')
    return text

# --- LLM CORRECTION PROMPT ---
SYSTEM_PROMPT = """You are an expert in early modern Spanish historical documents from the 17th century.
Your task is to correct OCR errors in transcribed text from historical printed sources.
```

```
STRICT RULES FOR 17TH CENTURY SPANISH:

1. INTERCHANGEABLE CHARACTERS:
   - 'u' and 'v': use 'u' at the beginning of a word, 'v' inside a word
     Example: 'vna' → 'una', 'avnque' → 'aunque'
   - 'f' and 's': use 's' at the beginning/end of a word, 'f' within a word
     Example: 'fus' → 'sus', 'defde' → 'desde'
   - 'i' and 'j': these are interchangeable, preserved as OCR detected

2. TILDE RULES:
   - q with tilde (q̃) → always expand to 'que'
   - vowel with tilde → vowel + n (ã→an, ẽ→en, ĩ→in, õ→on, ũ→un)
   - n with tilde → always ñ

3. OLD SPELLINGS:
   - ç → always modern z (cobrança → cobrança, keep as z)
   - long-s (ſ) → s

4. DO NOT:
   - Add or remove words
   - Modernize vocabulary
   - Change Latin phrases
   - Add accents that are not there
   - Change period-appropriate spelling variations

5. ONLY fix clear OCR character errors:
   - rn → m, cl → d, li → h
   - 0 → o, 1 → l, 5 → s
   - Missing spaces between words

Return ONLY the corrected text, nothing else."""

def llm_correct(text, max_retries=3):
    """Correct OCR text using Gemini."""
    prompt = f"""Correct the OCR errors in this 17th century Spanish text. Follow the rules strictly.

OCR TEXT:
{text}

CORRECTED TEXT:"""
```

```
    for attempt in range(max_retries):
        try:
            response = model.generate_content(
                prompt,
                generation_config=genai.GenerationConfig(
                    temperature=0.0,  # low temperature for deterministic correction
                    max_output_tokens=256,
                )
            )
            return response.text.strip()
        except Exception as e:
            print(f"Attempt {attempt+1} failed: {e}")
            time.sleep(2)
    return text  # return original if all retries fail

# --- LOAD TEST RESULTS ---
with open(TEST_RESULTS_PATH, 'r', encoding='utf-8') as f:
    test_data = json.load(f)

results = test_data['results']
print(f"Processing {len(results)} predictions...")

# --- APPLY RULE-BASED + LLM CORRECTION ---
cer_metric = evaluate.load("cer")
wer_metric = evaluate.load("wer")

corrected_results = []
preds_before = []
preds_after_rules = []
preds_after_llm = []
refs = []

for i, result in enumerate(results):
    original_pred = result['predicted']
    expected = result['expected']

    # Step 1 — Rule-based normalization
    rule_corrected = rule_based_normalize(original_pred)

    # Step 2 — LLM correction
```

```
    llm_corrected = llm_correct(rule_corrected)

    preds_before.append(original_pred.lower())
    preds_after_rules.append(rule_corrected.lower())
    preds_after_llm.append(llm_corrected.lower())
    refs.append(expected.lower())

    corrected_results.append({
        "image": result['image'],
        "expected": expected,
        "predicted_raw": original_pred,
        "predicted_rules": rule_corrected,
        "predicted_llm": llm_corrected,
        "source": result.get('source', ''),
        "page": result.get('page', '')
    })

    # Rate limiting — Gemini free tier: 15 req/min
    if (i + 1) % 14 == 0:
        print(f"Processed {i+1}/{len(results)} — waiting 60s for rate limit...")
        time.sleep(60)

    if (i + 1) % 10 == 0:
        print(f"Progress: {i+1}/{len(results)}")

# --- COMPUTE METRICS ---
cer_before = cer_metric.compute(predictions=preds_before, references=refs)
cer_after_rules = cer_metric.compute(predictions=preds_after_rules, references=refs)
cer_after_llm = cer_metric.compute(predictions=preds_after_llm, references=refs)

wer_before = wer_metric.compute(predictions=preds_before, references=refs)
wer_after_llm = wer_metric.compute(predictions=preds_after_llm, references=refs)

print(f"\n{'='*60}")
print(f"RESULTS SUMMARY")
print(f"{'='*60}")
print(f"CER before correction:       {cer_before:.4f} ({cer_before*100:.2f}%)")
print(f"CER after rule-based:        {cer_after_rules:.4f} ({cer_after_rules*100:.2f}%)")
print(f"CER after LLM correction:    {cer_after_llm:.4f} ({cer_after_llm*100:.2f}%)")
print(f"WER before correction:       {wer_before:.4f} ({wer_before*100:.2f}%)")
print(f"WER after LLM correction:    {wer_after_llm:.4f} ({wer_after_llm*100:.2f}%)")
```

```
print(f"Delta CER (LLM improvement): {(cer_before - cer_after_llm):.4f} ({(cer_before - cer_after_llm)*100:.2f}%)")
print(f"{'='*60}")

# Sample outputs
print(f"\n=== Sample Corrections ===")
for r in corrected_results[:5]:
    print(f"Expected:    '{r['expected']}'")
    print(f"Raw OCR:     '{r['predicted_raw']}'")
    print(f"After rules: '{r['predicted_rules']}'")
    print(f"After LLM:   '{r['predicted_llm']}'")
    print()

# Save results
with open(LLM_RESULTS_PATH, 'w', encoding='utf-8') as f:
    json.dump({
        "cer_before": cer_before,
        "cer_after_rules": cer_after_rules,
        "cer_after_llm": cer_after_llm,
        "wer_before": wer_before,
        "wer_after_llm": wer_after_llm,
        "delta_cer": cer_before - cer_after_llm,
        "total_pairs": len(results),
        "results": corrected_results
    }, f, ensure_ascii=False, indent=2)

print(f"Results saved to: {LLM_RESULTS_PATH}")
```

```
/usr/local/lib/python3.12/dist-packages/wrapt/importer.py:223: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  self.__wrapped__.exec_module(module)
```

```
Processing 195 predictions...
Progress: 10/195
Processed 14/195 — waiting 60s for rate limit...
Progress: 20/195
Processed 28/195 — waiting 60s for rate limit...
Progress: 30/195
Progress: 40/195
Processed 42/195 — waiting 60s for rate limit...
Progress: 50/195
Processed 56/195 — waiting 60s for rate limit...
Progress: 60/195
Processed 70/195 — waiting 60s for rate limit...
Progress: 70/195
Progress: 80/195
Processed 84/195 — waiting 60s for rate limit...
Progress: 90/195
Processed 98/195 — waiting 60s for rate limit...
Progress: 100/195
Progress: 110/195
Processed 112/195 — waiting 60s for rate limit...
Progress: 120/195
Processed 126/195 — waiting 60s for rate limit...
Progress: 130/195
Processed 140/195 — waiting 60s for rate limit...
Progress: 140/195
Progress: 150/195
Processed 154/195 — waiting 60s for rate limit...
Progress: 160/195
Processed 168/195 — waiting 60s for rate limit...
Progress: 170/195
Progress: 180/195
Processed 182/195 — waiting 60s for rate limit...
Progress: 190/195

============================================================
RESULTS SUMMARY
============================================================
CER before correction:       0.1133 (11.33%)
CER after rule-based:        0.1132 (11.32%)
CER after LLM correction:    0.1205 (12.05%)
WER before correction:       0.2740 (27.40%)
```

```
WER after LLM correction:    0.2699 (26.99%)
Delta CER (LLM improvement): -0.0072 (-0.72%)
============================================================

=== Sample Corrections ===
Expected:    'yor parte.'
Raw OCR:     'yor parte. No.'
After rules: 'yor parte. No.'
After LLM:   'yor parte. No.'

Expected:    'alomenos concurrir con ella: y que de aqui infirio, que pues'
Raw OCR:     'alomenos concurrir con ella: y de aqui infirió, que pues'
After rules: 'alomenos concurrir con ella: y de aqui infirió, que pues'
After LLM:   'alomenos concurrir con ella: y de aquí infirió, que pues'

Expected:    'hallarse subdita de su marido, y prohibida a acusalle: y'
Raw OCR:     'hallarse sabdita de su marido, y prohibida a acusalla: y'
After rules: 'hallarse sabdita de su marido, y prohibida a acusalla: y'
After LLM:   'hallarse súbdita de su marido, y prohibida a acusalla: y'

Expected:    '& 3. ibi: Primo scilicet uxor, secundo pater, tertio liberi'
Raw OCR:     'de 3. ibi. Primos silicet vxor, secundo pater, testio liberi'
After rules: 'de 3. ibi. Primos silicet vxor, secundo pater, testio liberi'
After LLM:   'de 3. ibi. Primos scilicet vxor, secundo pater, tertio liberi'

Expected:    'ua, y ratifica. Y ansimismo declara, que tiene otor'
Raw OCR:     'ua, y ratifica. Y ansimismo declara, que tiene otor'
After rules: 'ua, y ratifica. Y ansimismo declara, que tiene otor'
After LLM:   'ua, y ratifica. Y ansimismo declara, que tiene otor'

Results saved to: /kaggle/working/llm_results.json
```

Full Pipeline during inference(Gradio App)

1. Layout Analysis: Detect if the image is a single-page or two-page scan using vertical pixel projection and split it if a central white valley is found.
2. Geometric Filtering: Apply a "geometric fence" based on median edges to remove marginalia while preserving the sparse main text.

3. Noise Removal: Filter out small dots, scanner artifacts, and decorative symbols (like top-page crosses) using aspect ratio and density checks.
4. Line Detection: Use the CRAFT model to extract precise bounding boxes for every individual line of text.
5. Neural Transcription: Input each line crop into a fine-tuned TrOCR model to generate raw text strings.
6. Catchword Filtering: Identify and remove isolated words or hyphenated fragments (like "crian-") at the bottom of the column.
7. Line Concatenation: Join the cleaned lines from each column into a single structured raw text block.
8. VLM Post-Processing: Pass the image and raw text to Gemini to identify and prepend decorative Drop-cap letters.Orthographic Correction:
9. Use the VLM to fix character swaps (e.g., $v \rightarrow u$) while strictly forbidding word-joining or text reconstruction.
10. Also, CRAFT was splitting sentences (especially headers) into two and jumbling, because it had too many gaps in between. So that is also fixed using VLM.
11. Apart from that, marginal texts that are very close to the main text don't get filtered using the geometric filtering, so that is handled by VLM.
12. The removal of the last letter in the prompt to the VLM is given because, if hallucinates and adds, the word back. Otherwise, the Catchword Filtering is working for removal of last word.

In [ ]:
```
import nest_asyncio
nest_asyncio.apply()
import gradio as gr
import os
import gc
import cv2
import torch
import json
import numpy as np
import torchvision.models.vgg as vgg
from PIL import Image
from transformers import TrOCRProcessor, VisionEncoderDecoderModel, GenerationConfig
import google.generativeai as genai
vgg.model_urls = {
    'vgg16_bn': 'https://download.pytorch.org/models/vgg16_bn-6c64b313.pth'
}

from craft_text_detector import (
    read_image,
```

```
    load_craftnet_model,
    load_refinenet_model,
    get_prediction,
    empty_cuda_cache
)

# --- CONFIG ---
TROCR_MODEL_PATH = "/kaggle/input/models/indrasn0wal/vanilla-trocr-real-checkpoint/pytorch/default/1"
GEMINI_API_KEY = "your-api-key"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
PADDING = 8

# ============================================================
# LOAD MODELS ONCE
# ============================================================
print("Loading TrOCR...")
processor = TrOCRProcessor.from_pretrained(TROCR_MODEL_PATH)
trocr_model = VisionEncoderDecoderModel.from_pretrained(TROCR_MODEL_PATH)
trocr_model.config.decoder_start_token_id = processor.tokenizer.cls_token_id
trocr_model.config.pad_token_id = processor.tokenizer.pad_token_id
trocr_model.config.eos_token_id = processor.tokenizer.sep_token_id
trocr_model.generation_config = GenerationConfig(
    decoder_start_token_id=processor.tokenizer.cls_token_id,
    pad_token_id=processor.tokenizer.pad_token_id,
    eos_token_id=processor.tokenizer.sep_token_id,
    max_new_tokens=64,
    num_beams=4,
)
trocr_model.to(DEVICE)
trocr_model.eval()
print("TrOCR loaded ✅")

print("Loading CRAFT...")
craft_net = load_craftnet_model(cuda=torch.cuda.is_available())
refine_net = load_refinenet_model(cuda=torch.cuda.is_available())
print("CRAFT loaded ✅")

print("Loading Gemini...")
genai.configure(api_key=GEMINI_API_KEY)
gemini_model = genai.GenerativeModel("gemini-2.5-flash")
print("Gemini loaded ✅")
```

```python
# ============================================================
# PIPELINE FUNCTIONS
# ============================================================

def preprocess_remove_margins(image, margin_ratio=0.07):
    """Remove hard outer margins — scanner edges, blank borders."""
    height, width, _ = image.shape
    left_crop = int(width * margin_ratio)
    right_crop = int(width * (1 - margin_ratio))
    top_crop = int(height * 0.03)
    bottom_crop = int(height * 0.97)
    cropped = image[top_crop:bottom_crop, left_crop:right_crop]
    return cropped, left_crop, top_crop


def detect_two_column_projection(image):
    """Detect two-column layout using vertical projection. No CRAFT needed."""
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    _, thresh = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
    vertical_proj = np.sum(thresh, axis=0)
    height, width = gray.shape

    search_start = int(width * 0.35)
    search_end = int(width * 0.65)
    search_region = vertical_proj[search_start:search_end]

    max_val = np.max(search_region)
    min_val = np.min(search_region)
    if max_val == min_val:
        return False, None

    normalized = (search_region - min_val) / (max_val - min_val)

    # Need sustained deep white valley — at least 20 consecutive columns
    valley_threshold = 0.95
    high_white = normalized > valley_threshold
    count = 0
    max_count = 0
    for val in high_white:
        if val:
```

```
            count += 1
            max_count = max(max_count, count)
        else:
            count = 0

    if max_count >= 20:
        split_col = search_start + np.argmax(search_region)
        return True, split_col

    return False, None


def detect_craft_boxes(image_segment):
    """Run CRAFT on an image segment."""
    prediction_result = get_prediction(
        image=image_segment,
        craft_net=craft_net,
        refine_net=refine_net,
        text_threshold=0.7,
        link_threshold=0.4,
        low_text=0.4,
        cuda=torch.cuda.is_available(),
        long_size=1280,
        poly=False
    )
    return prediction_result["boxes"]


def filter_boxes_basic(image_segment, raw_boxes, x_offset=0, y_offset=0):
    """
    Basic geometric filter — removes obvious noise and decorations.
    Does NOT filter sidenotes (handled separately by positional filter).
    """
    height, width = image_segment.shape[:2]

    accepted = []
    rejected = []

    for box in raw_boxes:
        x_coords = [pt[0] for pt in box]
        y_coords = [pt[1] for pt in box]
```

```
        cx = np.mean(x_coords)
        cy = np.mean(y_coords)
        w = max(x_coords) - min(x_coords)
        h = max(y_coords) - min(y_coords)
        aspect = w / h if h > 0 else 0

        # Hard top — page numbers, scanner artifacts
        # if cy < height * 0.03:
        #     rejected.append((box, 'top'))
        #     continue
        if cy < height * 0.05 and aspect < 1.5:
            continue

        # Too square — decorative initials, stamps
        # if aspect < 1.2:
        #     rejected.append((box, f'square {aspect:.1f}'))
        #     continue

        # Too small — noise dots
        if w < 20 or h < 8:
            rejected.append((box, 'tiny'))
            continue

        # Density check — ink blobs or empty regions
        x1 = int(max(0, min(x_coords)))
        y1 = int(max(0, min(y_coords)))
        x2 = int(min(width, max(x_coords)))
        y2 = int(min(height, max(y_coords)))
        if x2 > x1 and y2 > y1:
            crop = image_segment[y1:y2, x1:x2]
            if crop.size > 0:
                gray_crop = cv2.cvtColor(crop, cv2.COLOR_RGB2GRAY) \
                    if len(crop.shape) == 3 else crop
                _, binary = cv2.threshold(
                    gray_crop, 128, 255, cv2.THRESH_BINARY_INV
                )
                density = np.sum(binary > 0) / binary.size
                if density < 0.02 or density > 0.80:
                    rejected.append((box, f'density {density:.3f}'))
                    continue
```

```
        # Restore to full image coordinates
        adjusted_box = [
            [pt[0] + x_offset, pt[1] + y_offset]
            for pt in box
        ]
        accepted.append(adjusted_box)

    return accepted, rejected

def filter_end_of_page_words(lines_data):
    """
    Runs AFTER TrOCR on a single column's data.
    Groups into lines. If the VERY LAST line of the column is just one word,
    or if the last word ends in a hyphen, it removes it.
    Lines in the middle of the paragraph/column are completely untouched.
    """
    if not lines_data:
        return lines_data

    # 1. Group into horizontal lines based on Y-center
    lines = []
    current_line = [lines_data[0]]

    heights = [item['bbox'][3] - item['bbox'][1] for item in lines_data]
    line_threshold = np.median(heights) * 0.5

    for item in lines_data[1:]:
        item_cy = (item['bbox'][1] + item['bbox'][3]) / 2.0
        line_cy = np.mean([(x['bbox'][1] + x['bbox'][3]) / 2.0 for x in current_line])

        if abs(item_cy - line_cy) < line_threshold:
            current_line.append(item)
        else:
            lines.append(current_line)
            current_line = [item]

    if current_line:
        lines.append(current_line)

    # TARGET ONLY THE LAST LINE OF THE COLUMN
    last_line = lines[-1]
```

```
    # 2. Apply rules ONLY to the very last line

    # Rule A: Last line of the column is exactly ONE box
    if len(last_line) == 1:
        text = last_line[0]['text'].strip()
        # If it's a single word (no space) OR ends with a hyphen, drop the whole line.
        if " " not in text or text.endswith("-"):
            lines = lines[:-1]

    # Rule B: Last line has multiple words, check the right-most word
    else:
        last_line = sorted(last_line, key=lambda x: x['bbox'][0])
        last_word_text = last_line[-1]['text'].strip()

        if last_word_text.endswith("-"):
            last_line.pop() # Remove only the last word
            lines[-1] = last_line

    # 3. Flatten back into a single list
    filtered_data = []
    for line in lines:
        filtered_data.extend(line) # Middle lines are added back safely here!

    return filtered_data

def filter_sidenotes_positional(accepted_boxes):
    """
    Filter sidenotes using WIDTH + BOUNDARIES combined.
    Prevents accidentally deleting short paragraph endings (like "Amen.")
    """
    if len(accepted_boxes) < 4: return accepted_boxes, []

    # Identify the main text column boundaries
    left_edges = [min([pt[0] for pt in b]) for b in accepted_boxes]
    right_edges = [max([pt[0] for pt in b]) for b in accepted_boxes]
    col_left = np.median(left_edges)
    col_right = np.median(right_edges)
    median_w = np.median([r - l for l, r in zip(left_edges, right_edges)])

    main_text = []
```

```
    sidenotes = []
    # Reduced tolerance: 8% instead of 12% to catch citations like '8.'
    tolerance = median_w * 0.08

    for box in accepted_boxes:
        b_left = min([pt[0] for pt in box])
        b_right = max([pt[0] for pt in box])
        w = b_right - b_left

        is_narrow = w < median_w * 0.40
        # If the box is entirely to the right of the paragraph end or left of the start
        is_outside = (b_right < col_left + tolerance) or (b_left > col_right - tolerance)

        if is_narrow and is_outside:
            sidenotes.append(box)
        else:
            main_text.append(box)
    return main_text, sidenotes


def merge_fragmented_headlines(boxes, x_threshold=150, y_threshold=20):
    """
    Rejoins headlines and sentences split by wide spacing.
    Fixed numpy array handling for variable point counts.
    """
    if not boxes:
        return []

    boxes = sorted(
        boxes,
        key=lambda b: (np.mean([pt[1] for pt in b]),
                       min([pt[0] for pt in b]))
    )

    merged = []
    while boxes:
        curr = boxes.pop(0)
        found = False

        for i, m_box in enumerate(merged):
            y_dist = abs(
```

```
                np.mean([pt[1] for pt in curr]) -
                np.mean([pt[1] for pt in m_box])
            )
            curr_left = min([pt[0] for pt in curr])
            m_right = max([pt[0] for pt in m_box])
            x_gap = curr_left - m_right

            # if y_dist < y_threshold and 0 <= x_gap < x_threshold:
            if y_dist < y_threshold and -20 <= x_gap < x_threshold:
                # Merge - fixed numpy handling
                all_x = [pt[0] for pt in m_box] + [pt[0] for pt in curr]
                all_y = [pt[1] for pt in m_box] + [pt[1] for pt in curr]
                new_box = [
                    [min(all_x), min(all_y)],
                    [max(all_x), min(all_y)],
                    [max(all_x), max(all_y)],
                    [min(all_x), max(all_y)]
                ]
                merged[i] = new_box
                found = True
                break

        if not found:
            merged.append(curr)

    return merged


def crop_line_with_padding(image, box, padding=PADDING):
    """Crop line from full image with padding."""
    x_coords = [pt[0] for pt in box]
    y_coords = [pt[1] for pt in box]
    x1 = int(max(0, min(x_coords) - padding))
    y1 = int(max(0, min(y_coords) - padding))
    x2 = int(min(image.shape[1], max(x_coords) + padding))
    y2 = int(min(image.shape[0], max(y_coords) + padding))
    crop = image[y1:y2, x1:x2]
    return crop, (x1, y1, x2, y2)


def trocr_predict(crop_bgr):
```

```
    """Run TrOCR on a single line crop."""
    if len(crop_bgr.shape) == 3 and crop_bgr.shape[2] == 3:
        crop_rgb = cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2RGB)
    else:
        crop_rgb = crop_bgr
    pil_img = Image.fromarray(crop_rgb).convert("RGB")
    pixel_values = processor(
        pil_img, return_tensors="pt"
    ).pixel_values.to(DEVICE)
    with torch.no_grad():
        generated = trocr_model.generate(
            pixel_values,
            generation_config=trocr_model.generation_config
        )
    return processor.batch_decode(generated, skip_special_tokens=True)[0].strip()


def gemini_postprocess_v2(raw_text, pil_image):
    """VLM correction — runs once on full page image + raw text."""
    prompt = """You are an expert 17th-century Spanish editor specializing in faithful transcriptions.

I am providing a scan and its raw OCR transcription.

THE CHALLENGE:
The transcription contains three distinct zones:
1. HEADERS/TITLES: Centered text at the very top
2. THE DROP CAP: A massive decorative woodcut letter
3. THE BODY PROSE: The main paragraphs

YOUR CRITICAL INSTRUCTION:
Identify the decorative woodcut 'Drop Cap' letter in the IMAGE. It is located
at the start of the main body paragraph, NOT the centered headers.
Prepend this letter ONLY to the first word of the main body prose.

TASK 2 — JUMBLED / SPLIT LINES:
Some lines were detected as separate fragments due to gaps in the printed text.
Look at the image and fix such splits by reconstructing the correct line.

TASK 3 — INCOMPLETE PAGE ENDING:
If the last word or line is single word, half word,
lone hyphen, or catchword continuing on next page, remove it entirely. And if it is not present in the text, don't
```

```
TASK 4 — ORTHOGRAPHY FIXES:
- u at start of word, v inside: vna→una, avnque→aunque
- s at start/end of word, f within: fus→sus, defde→desde
- q with tilde → que
- vowel with tilde → vowel + n: ũ→un, ẽ→en, ã→an
- ç → z
- long-s (ſ) → s
Task 5 - No Additions:
Ignore all citations or notes in the image margins. If they are not in the RAW OCR, DO NOT add them back.

Task 6 - Remove any Decorative element:
Look at the image, and if any decorative element is in the text remove it.

STRICT CONSTRAINTS:
- DO NOT JOIN LINES: Keep every line break exactly as it appears in the RAW OCR TEXT.
- DO NOT include any decorative element.
- DO NOT modernize vocabulary or spelling
- DO NOT add accent marks not clearly in the original
- DO NOT change Latin phrases
- DO NOT add commentary or explanations
- If unsure, keep the original OCR text

Return ONLY the final corrected transcription."""
    try:
        response = gemini_model.generate_content([
            prompt,
            pil_image,
            f"RAW OCR TEXT:\n{raw_text}"
        ])
        return response.text.strip()
    except Exception as e:
        return f"[Gemini error: {e}]\n\n{raw_text}"


# ============================================================
# MAIN PIPELINE
# ============================================================

def run_pipeline(pil_image, apply_gemini):
    if pil_image is None:
```

```
        return None, "", "", "No image provided."

    log = []
    image = np.array(pil_image.convert("RGB"))
    image_bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    height, width, _ = image.shape
    log.append(f"Image size: {width}x{height}")

    # Step 1 — Remove hard outer margins
    image_cropped, x_offset, y_offset = preprocess_remove_margins(
        image, margin_ratio=0.07
    )
    log.append(f"Margin crop: x_offset={x_offset}, y_offset={y_offset}")

    # Step 2 — Layout detection on cropped image
    is_two_col, split_x_cropped = detect_two_column_projection(image_cropped)
    layout = "Two-column" if is_two_col else "Single-column"
    log.append(f"Layout: {layout}")

    # Step 3 — Build columns
    if is_two_col:
        split_x_full = split_x_cropped + x_offset
        left_col = image_cropped[:, :split_x_cropped]
        right_col = image_cropped[:, split_x_cropped:]
        log.append(f"Split at x={split_x_full}")
        cols = [
            (left_col,  x_offset,     y_offset, 'left'),
            (right_col, split_x_full, y_offset, 'right'),
        ]
    else:
        cols = [(image_cropped, x_offset, y_offset, 'single')]

    all_lines_data = []
    all_sidenote_boxes = []
    all_rejected_boxes = []

    for col_img, x_off, y_off, col_name in cols:
        log.append(f"Processing '{col_name}'...")

        # CRAFT — once per column
        col_boxes = detect_craft_boxes(col_img)
```

```
        log.append(f"  CRAFT boxes: {len(col_boxes)}")

        # Basic filter — noise, decoratives, tiny boxes
        accepted, rejected = filter_boxes_basic(
            col_img, col_boxes,
            x_offset=x_off,
            y_offset=y_off
        )
        all_rejected_boxes.extend(rejected)
        log.append(f"  After basic filter: {len(accepted)} accepted, "
                   f"{len(rejected)} rejected")

        # Positional sidenote filter — narrow AND far from center
        accepted, sidenotes = filter_sidenotes_positional(accepted)
        all_sidenote_boxes.extend(sidenotes)
        log.append(f"  After sidenote filter: {len(accepted)} main, "
                   f"{len(sidenotes)} sidenotes")

        # Merge fragmented headlines
        accepted = merge_fragmented_headlines(accepted)
        log.append(f"  After headline merge: {len(accepted)} lines")

        # Sort top to bottom
        accepted.sort(key=lambda b: np.mean([pt[1] for pt in b]))
        col_lines_data = []

        # TrOCR per line
        for box in accepted:
            crop, coords = crop_line_with_padding(image_bgr, box)
            if crop.size == 0 or crop.shape[0] < 5 or crop.shape[1] < 15:
                continue
            text = trocr_predict(crop)
            col_lines_data.append({
                "text": text,
                "bbox": list(coords),
                "column": col_name
            })
        original_count = len(col_lines_data)
        col_lines_data = filter_end_of_page_words(col_lines_data)
        if len(col_lines_data) < original_count:
            log.append(f"  Dropped {original_count - len(col_lines_data)} trailing words/hyphens.")
```

```
        all_lines_data.extend(col_lines_data)

    log.append(f"Total lines transcribed: {len(all_lines_data)}")

    # Concatenate
    raw_text = "\n".join([l["text"] for l in all_lines_data])

    # Gemini — once on full page
    if apply_gemini and raw_text.strip():
        log.append("Running Gemini VLM...")
        final_text = gemini_postprocess_v2(raw_text, pil_image)
    else:
        final_text = raw_text
        log.append("Gemini skipped.")

    # Visualization
    vis = image_bgr.copy()

    # Blue — margin boundaries
    cv2.line(vis, (x_offset, 0), (x_offset, height), (255, 0, 0), 2)
    cv2.line(vis, (width - x_offset, 0),
             (width - x_offset, height), (255, 0, 0), 2)

    # Red — column split
    if is_two_col:
        cv2.line(vis, (split_x_full, 0),
                 (split_x_full, height), (0, 0, 255), 3)

    # Purple — sidenotes
    for box in all_sidenote_boxes:
        pts = np.array(
            [[int(pt[0]), int(pt[1])] for pt in box], dtype=np.int32
        )
        cv2.polylines(vis, [pts], isClosed=True,
                      color=(255, 0, 255), thickness=1)

    # Dark red — basic rejected
    for box, reason in all_rejected_boxes:
        pts = np.array(
            [[int(pt[0]), int(pt[1])] for pt in box], dtype=np.int32
```

```
        )
        cv2.polylines(vis, [pts], isClosed=True,
                      color=(0, 0, 180), thickness=1)

    # Green / orange — accepted main text
    for line in all_lines_data:
        x1, y1, x2, y2 = line['bbox']
        color = (0, 200, 0) if line['column'] != 'right' else (0, 165, 255)
        cv2.rectangle(vis, (x1, y1), (x2, y2), color, 2)

    annotated_pil = Image.fromarray(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))

    empty_cuda_cache()
    gc.collect()

    return annotated_pil, raw_text, final_text, "\n".join(log)


# ============================================================
# GRADIO APP
# ============================================================

def gradio_fn(image, apply_gemini):
    if image is None:
        return None, "", "", "Please upload an image."
    try:
        pil_image = Image.fromarray(image) \
            if isinstance(image, np.ndarray) else image
        return run_pipeline(pil_image, apply_gemini)
    except Exception as e:
        import traceback
        return None, "", f"Error: {str(e)}", traceback.format_exc()


with gr.Blocks(title="RenAIssance OCR", theme=gr.themes.Soft()) as demo:

    gr.Markdown("""
    # 📜 RenAIssance OCR Pipeline
    ### Automated transcription of 17th century Spanish printed documents
    **Pipeline:** Margin crop → Layout detection → CRAFT → Basic filter → Positional sidenote filter → Headline mer
    """)
```

```
    with gr.Row():
        with gr.Column(scale=1):
            gr.Markdown("### 📤 Input Image")
            input_image = gr.Image(
                label="Upload Page",
                type="pil",
                height=700
            )
            apply_gemini = gr.Checkbox(
                label="Apply Gemini VLM post-correction",
                value=True
            )
            run_btn = gr.Button(
                "🔍 Transcribe Page",
                variant="primary",
                size="lg"
            )

        with gr.Column(scale=1):
            gr.Markdown("### 🖼️ Detected Lines")
            output_image = gr.Image(
                label="Blue=margins | Red=col split | Green=main | Orange=right | Purple=sidenotes",
                height=700
            )

        with gr.Column(scale=1):
            gr.Markdown("### 📝 Transcription")
            with gr.Tabs():
                with gr.Tab("✅ Final (Gemini corrected)"):
                    final_text_box = gr.Textbox(
                        label="Final Transcription",
                        lines=28,
                        max_lines=50,
                        show_copy_button=True
                    )
                with gr.Tab("🔤 Raw (TrOCR only)"):
                    raw_text_box = gr.Textbox(
                        label="Raw TrOCR Output",
                        lines=28,
                        max_lines=50,
```

```
                        show_copy_button=True
                    )

    info_box = gr.Textbox(
        label="📊 Pipeline Log",
        interactive=False,
        lines=12
    )

    run_btn.click(
        fn=gradio_fn,
        inputs=[input_image, apply_gemini],
        outputs=[output_image, raw_text_box, final_text_box, info_box]
    )

    gr.Markdown("""
    ---
    **RenAIssance GSoC 2025** | CRAFT + TrOCR + Gemini VLM
    """)


demo.launch(
    share=True,
    debug=True,
    server_name="0.0.0.0",
    server_port=7860
)
```

```
/usr/local/lib/python3.12/dist-packages/wrapt/importer.py:223: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  self.__wrapped__.exec_module(module)
Loading TrOCR...
Loading weights:   0%|          | 0/480 [00:00<?, ?it/s]
TrOCR loaded ✅
Loading CRAFT...
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is
deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight
enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equ
ivalent to passing `weights=None`.
  warnings.warn(msg)
Craft text detector weight will be downloaded to /root/.craft_text_detector/weights/craft_mlt_25k.pth
Downloading...
From: https://drive.google.com/uc?id=1bupFXqT-VU6Jjeul13XP7yx2Sg5IHr4J
To: /root/.craft_text_detector/weights/craft_mlt_25k.pth
100%|██████████| 83.2M/83.2M [00:00<00:00, 255MB/s]
Craft text refiner weight will be downloaded to /root/.craft_text_detector/weights/craft_refiner_CTW1500.pth
Downloading...
From: https://drive.google.com/uc?id=1xcE9qpJXp4ofINwXWVhhQIh9S8Z7cuGj
To: /root/.craft_text_detector/weights/craft_refiner_CTW1500.pth
100%|██████████| 1.85M/1.85M [00:00<00:00, 186MB/s]
/tmp/ipykernel_55/1135122139.py:529: DeprecationWarning: The 'theme' parameter in the Blocks constructor will be rem
oved in Gradio 6.0. You will need to pass 'theme' to Blocks.launch() instead.
  with gr.Blocks(title="RenAIssance OCR", theme=gr.themes.Soft()) as demo:
CRAFT loaded ✅
Loading Gemini...
Gemini loaded ✅
* Running on local URL:  http://0.0.0.0:7860
* Running on public URL: https://186f5b6f0d123b0d37.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the termina
l in the working directory to deploy to Hugging Face Spaces (https://huggingface.co/spaces)
```

gradio

No interface is running right now

```
Keyboard interruption in main thread... closing server.
Killing tunnel 0.0.0.0:7860 <> https://186f5b6f0d123b0d37.gradio.live
```